

Fondations I

Du neurone à la régression logistique

Pourquoi le deep learning · rappels mathématiques · régression logistique ·
vectorisation NumPy · PCA

Journée 1 · jeudi 08/10/2026 · 9h00–13h00 / 14h00–17h00



Où en sommes-nous ?

9 journées · 63 h · projet fil rouge soutenu le 05/02/2027



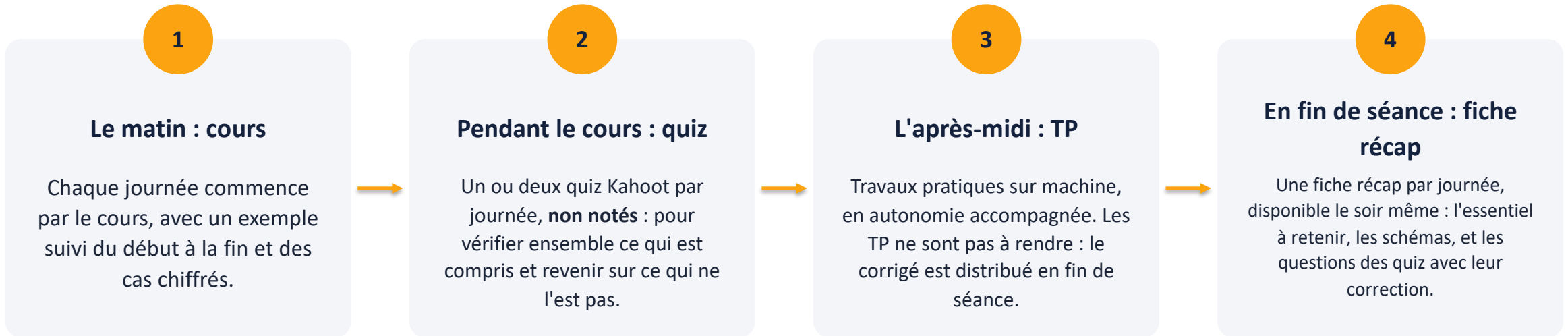
Aujourd'hui · J1 · 08/10/26

Fondations I — neurone → régression logistique

Programme de la journée : 4 heures de cours, 3 heures de TP

1	9h00	Présentation du cours (15 min)	Calendrier, façon de travailler, évaluation ; le projet sera présenté à la journée 2
2	9h15	Un projet de machine learning (35 min)	Supervisé et non supervisé ; les onze blocs d'un projet ; métrique ; le moteur d'un modèle
3	9h50	Pourquoi le deep learning ? (50 min)	Situer le DL, l'exemple des appartements en six étapes, le neurone, les réseaux
4	10h40	Quiz 1, puis pause	Dix questions non notées (8 min) ; pause de 15 min
5	11h00	Rappels mathématiques ciblés (15 min)	Produit matriciel, dérivée, règle de la chaîne
6	11h15	Régression logistique (50 min)	Modèle, coût, descente de gradient, graphe de calcul ; quiz 2 à la fin
7	12h05	Vectorisation et NumPy (30 min)	Broadcasting, pièges, boîte à outils
8	12h35	Explorer les données, ACP (15 min)	Checklist d'exploration, ACP en quatre étapes
9	12h50	Le TP de l'après-midi et l'essentiel	Présentation du TP1, ce qu'il faut retenir
10	14h00	TP1 (3h)	NumPy avancé · exploration + PCA · régression logistique from scratch ; corrigé en fin de séance

Comment nous allons travailler



- La seule note du module est celle du projet, mené en binôme tout au long du semestre et soutenu le 5 février. Il sera présenté à la journée 2.
- Le support de cours, le notebook de TP et la fiche récap de chaque journée sont mis à disposition sur l'espace du cours.

1

Un projet de machine learning, du besoin à la production

Le cadre dans lequel le deep learning s'inscrit : les mêmes blocs qu'un projet de machine learning classique

Apprentissage supervisé, non supervisé : de quoi parle-t-on ?

Supervisé : on connaît la réponse

- Chaque exemple vient avec sa **réponse** y ; le modèle apprend à la retrouver.
- **Régression** : prédire un nombre. Le prix d'un appartement, la consommation électrique de demain.
- **Classification binaire** : oui ou non. Un chat sur la photo, un client qui résilie.
- **Classification multi-classe** : une catégorie parmi plusieurs. Dix types de vêtements, un chiffre manuscrit.
- **Séries temporelles** : prédire la suite d'une séquence (journée 3).

Non supervisé : pas de réponse

- On ne dispose que des x ; le modèle cherche une **structure**.
- **Clustering** : regrouper. Des segments de clients, des familles de produits.
- **Réduction de dimension** : résumer. L'ACP, cet après-midi.
- **Détection d'anomalies** : repérer ce qui ne ressemble à rien. Une transaction frauduleuse (journée 8).
- **Génération** : produire de nouveaux exemples. Images, textes (journées 8 et 9).

Et le deep learning ?

- Presque toutes les applications industrielles rentables du ML sont **supervisées** (journées 1 à 7).
- Les grands modèles de langage sont **auto-supervisés** : la réponse est le mot suivant du texte, donc aucune annotation manuelle n'est nécessaire.
- Dans ce cours : supervisé jusqu'à la journée 7, non supervisé et génératif aux journées 8 et 9.
- Votre projet : une **régression** sur une **série temporelle**.

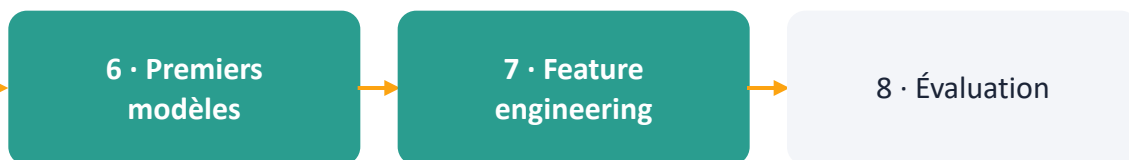
Les onze blocs d'un projet de machine learning

Le deep learning ne change que deux blocs ; tout le reste est le métier du data scientist

POSER LE PROBLÈME ET LE PLANCHER À BATTRE · blocs 1 à 5

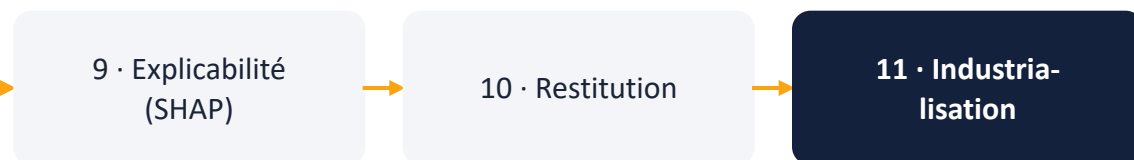


MODÉLISER · blocs 6 à 8, en boucle



on itère entre 6, 7 et 8 jusqu'à battre la baseline

LIVRER · blocs 9 à 11



 *deep learning intervient ici*

- Les blocs 1 à 5 posent le problème et le plancher à battre ; les blocs 6 à 8 tournent en boucle ; les blocs 9 à 11 rendent le résultat utile et durable.
- Le deep learning n'intervient qu'aux blocs 6 et 7 (en vert) : le réseau est un modèle parmi d'autres, qui apprend une partie de ses variables lui-même. Les neuf autres blocs sont identiques à un projet classique.
- Votre projet suit exactement ces blocs : besoin (les acheteurs d'énergie), métrique (l'écart moyen en MW), exploration, baseline, modèles comparés, évaluation, restitution (la soutenance) et industrialisation (le service sur Hugging Face).

Bloc 1 · Recueil du besoin

Qui décide, quelle décision, et combien coûte une erreur · Livrable : une fiche de cadrage d'une page

Ce qu'on fait

- Rencontrer les futurs utilisateurs : quelle **décision** le modèle doit-il aider, et qui la prend ?
- Décrire la situation actuelle : comment fait-on aujourd'hui, en combien de temps, avec quelle qualité ?
- Chiffrer le **coût d'une erreur** dans les deux sens, et les contraintes (délai, explicabilité, budget).

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- L'agent estime chaque bien au jugé, en 30 minutes, avec l'historique du quartier en tête.
- Un prix surestimé de 10 % : le bien reste trois mois invendu ; sous-estimé : le vendeur perd 20 k€ et change d'agence.
- Besoin : une estimation en moins d'une minute, fiable à 5 %, que l'agent peut expliquer au vendeur.

Bloc 2 · Identification de la métrique

Un nombre pour entraîner, un nombre pour décider · Livrable : une métrique à optimiser, des contraintes à respecter

Ce qu'on fait

- Choisir la **métrique d'apprentissage** : ce que le modèle minimise et ce qui départage deux versions.
- La relier à une **métrique métier** : ce que l'entreprise regarde réellement.
- Fixer les **contraintes** à respecter plutôt qu'à optimiser : temps de réponse, taille, explicabilité.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Métrique d'apprentissage : l'écart moyen en k€ entre prix estimé et prix de vente réel.
- Métrique métier : part des biens vendus en moins de 60 jours au prix annoncé.
- Contraintes : réponse en moins d'une seconde ; chaque estimation accompagnée de ses trois raisons principales.

Bloc 3 · Analyse exploratoire (EDA)

Regarder les données avant de modéliser · Livrable : un notebook d'exploration commenté

Ce qu'on fait

- Distributions, valeurs manquantes, valeurs aberrantes, doublons : connaître la matière première.
- Chercher les **fuites** : une colonne qui contient déjà la réponse (prix affiché, commission).
- Premières relations visibles : quelles variables expliquent déjà la cible ?

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- 12 000 ventes sur sept ans ; prix médian 245 k€ ; dix biens à plus de 2 M€ qui écrasent les graphiques.
- Le diagnostic énergétique manque pour 30 % des biens anciens ; 80 doublons entre deux exports.
- Le prix au m² varie du simple au double selon le quartier : le quartier sera une variable clé.

Bloc 4 · Preuve de concept (POC)

Répondre vite à une seule question : est-ce faisable ? · Livrable : un notebook et une décision go / no go

Ce qu'on fait

- Petit périmètre, données disponibles, deux semaines : on cherche un ordre de grandeur, pas un produit.
- Un modèle simple suffit : la question est « le signal existe-t-il ? ».
- Décision explicite à la fin : continuer, réorienter ou arrêter.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Deux semaines, un notebook, une régression sur la surface et le quartier.
- Écart moyen obtenu : 38 k€, contre 45 k€ pour l'estimation au jugé des agents.
- Décision : on continue, l'objectif de 5 % (environ 12 k€) semble atteignable avec plus de variables.

Bloc 5 · Création de la baseline

La règle la plus simple à battre · Livrable : un chiffre de référence sur le protocole d'évaluation

Ce qu'on fait

- Une règle de bon sens, sans apprentissage ou presque : moyenne, médiane par groupe, dernière valeur connue.
- Elle fixe le **plancher** : tout modèle qui ne la bat pas est inutile.
- Elle est souvent plus difficile à battre qu'on ne le croit.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Baseline : prix médian au m² du quartier × surface du bien.
- Écart moyen : 31 k€. Déjà meilleure que le POC.
- Toute la suite se compare à ces 31 k€, sur les mêmes ventes.

Bloc 6 · Premiers modèles

Plusieurs familles, un seul protocole · Livrable : un tableau comparatif, baselines en première ligne

Ce qu'on fait

- Essayer deux ou trois familles différentes, entraînées et évaluées **exactement de la même façon**.
- Le modèle le plus complexe n'est pas forcément le meilleur ; le plus simple qui atteint l'objectif gagne.
- Noter chaque essai : paramètres, données, résultat (c'est le rôle de MLflow, journée 4).

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Régression linéaire : 28 k€. Gradient boosting : 19 k€. Réseau de neurones dense : 21 k€.
- Même découpage, même métrique, même tableau : la comparaison est honnête.
- Le boosting mène ; le réseau apprend mal les quartiers avec peu de ventes.

Bloc 7 · Feature engineering

Donner au modèle les bonnes variables · Livrable : un jeu de variables documenté

Ce qu'on fait

- Créer, transformer, combiner des variables à partir de la connaissance du métier.
- Encoder proprement les catégories, les dates, les textes ; mettre les échelles en commun.
- En deep learning, le réseau apprend une partie de ces variables lui-même : c'est son grand intérêt.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Distance au tram, âge du bâtiment, étage avec ou sans ascenseur, prix au m² des cinq ventes voisines.
- Le boosting passe de 19 à 15 k€.
- Version réseau : le quartier devient un vecteur appris (un embedding), et le réseau rattrape le boosting.

Bloc 8 · Évaluation

Mesurer honnêtement, puis regarder où l'on se trompe · Livrable : un rapport d'évaluation par segment

Ce qu'on fait

- Un jeu de test jamais utilisé pour choisir quoi que ce soit ; un découpage temporel si le temps compte.
- La métrique globale, puis par segment : où le modèle est-il mauvais ?
- L'**analyse d'erreurs** décide de la prochaine action, au lieu de deviner.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Test : les ventes des six derniers mois. Écart moyen : 15 k€ ; objectif atteint.
- Mais 32 k€ sur les biens de plus de 150 m², et un biais : le modèle sous-estime les maisons avec jardin.
- Prochaine action : plus de variables sur l'extérieur, pas un modèle plus gros.

Bloc 9 · Explicabilité (SHAP)

Comprendre pourquoi le modèle dit ce qu'il dit · Livrable : trois raisons lisibles par estimation

Ce qu'on fait

- Pour chaque prédiction, la contribution de chaque variable : les valeurs de Shapley (bibliothèque SHAP).
- Sert à **vérifier** le modèle (apprend-il quelque chose de sensé ?), à le **déboguer** et à le faire **accepter**.
- Obligatoire dès qu'une décision touche une personne.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Estimation 312 k€ : surface + 85 k€ par rapport à la moyenne, DPE F – 12 k€, quatrième étage sans ascenseur – 9 k€.
- L'agent reconnaît son propre raisonnement : il fait confiance.
- On découvre que le modèle utilisait l'année de l'export comme variable : fuite corrigée.

Bloc 10 · Restitution

Parler aux décideurs dans leur langue · Livrable : une présentation de dix slides et une recommandation

Ce qu'on fait

- Partir du besoin, pas de l'architecture ; donner les résultats dans l'unité métier.
- Dire ce que le modèle ne sait pas faire, et ce qu'il faudrait pour aller plus loin.
- Terminer par une **recommandation** et les prochaines étapes.

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- Dix slides : écart moyen 15 k€ contre 45 k€ au jugé, délai de vente estimé réduit de trois semaines.
- Limites : grands biens et maisons avec jardin ; pas de données sur l'état intérieur.
- Recommandation : déployer sur les appartements, garder l'estimation humaine pour les maisons.

Bloc 11 · Industrialisation

Du notebook au service qui tourne tous les jours · Livrable : un service en ligne, surveillé et versionné

Ce qu'on fait

- Exposer le modèle par une API, le conteneuriser, l'intégrer dans l'outil des utilisateurs.
- Surveiller : qualité des prédictions dans le temps, dérive des données, pannes.
- Ré-entraîner selon un calendrier ou sur alerte, avec une version pour chaque modèle (journée 4).

Sur notre exemple : estimer le prix des biens d'une agence immobilière

- L'API est appelée depuis le logiciel de l'agence ; chaque estimation est enregistrée avec sa version.
- Tous les mois : comparaison estimation / prix de vente réel ; alerte si l'écart moyen dépasse 20 k€.
- Ré-entraînement trimestriel avec les nouvelles ventes.

La métrique : celle du modèle et celle de l'entreprise

Problème	Métrique d'apprentissage (ce que le modèle minimise)	Métrique métier (ce que l'entreprise regarde)	Contraintes à respecter
Estimer le prix d'un bien	écart moyen en k€	délai de vente, marge de l'agence	réponse en 1 s, explicable
Détecter une fraude	rappel à précision fixée	euros récupérés, clients honnêtes dérangés	décision en 100 ms
Prévoir la consommation électrique	écart moyen en MW sur 24 h	coût des écarts sur le marché	prévision livrée avant 6 h
Classer des photos de déchets	exactitude par classe	tonnes bien triées, temps d'opérateur	fonctionne sur un téléphone

Le cœur d'un modèle : une fonction de perte, et un moteur pour la faire baisser

Comparaison avec une voiture : ce qui compte n'est pas la carrosserie mais la destination, le GPS et le moteur



La destination

L'objectif : la métrique métier, puis la métrique d'apprentissage qui la traduit en nombre.



Le GPS : la fonction de perte

Elle dit en permanence à **quelle distance** on est de l'objectif, pour les paramètres actuels. C'est elle qui définit ce que le modèle cherche : changer de perte, c'est changer de destination.



Le moteur : la descente de gradient

Ce qui fait avancer : à chaque pas, elle regarde la pente de la perte et bouge les paramètres dans le sens qui la fait baisser. Même moteur de la régression logistique à GPT.



Le volant : les paramètres

w et b , puis des millions de poids : ce que le moteur ajuste. L'architecture (dense, convolutive, récurrente) est la carrosserie : elle change la forme, pas le principe.



Le carburant : les données

Sans exemples, rien n'avance ; avec des exemples biaisés, on va vite dans la mauvaise direction.



Le tableau de bord : la validation

Les métriques sur des données jamais vues : la seule façon de savoir si l'on avance vraiment ou si l'on tourne en rond (sur-apprentissage, journée 5).

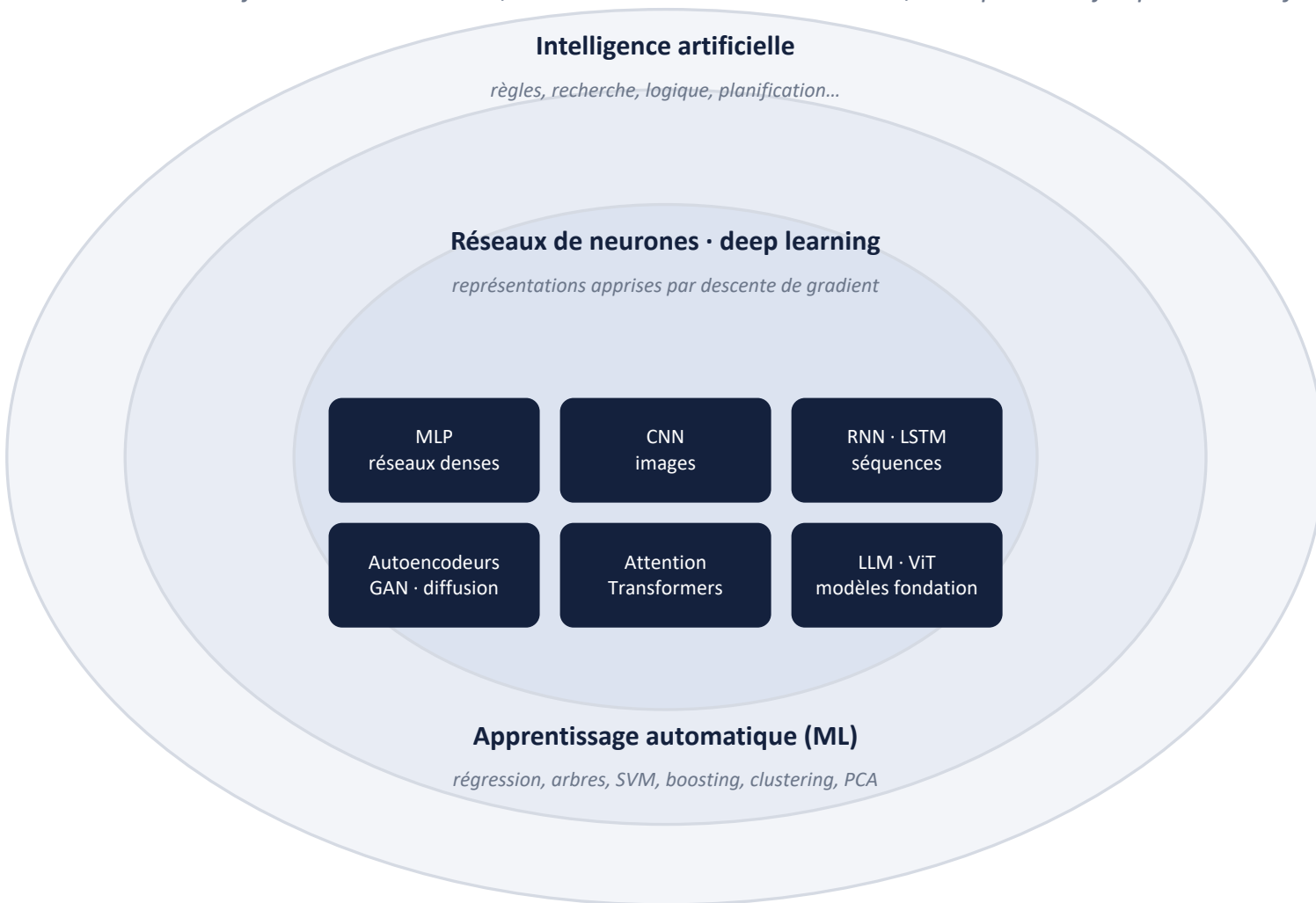
2

Pourquoi le deep learning ?

Le neurone comme brique, l'apprentissage supervisé comme cadre

Situer le deep learning : trois cercles imbriqués

Vous connaissez déjà le cercle du milieu ; ce cours entre dans le troisième, et le parcourt jusqu'aux Transformers



- **IA** : tout programme qui imite une capacité humaine — dont des systèmes à règles sans apprentissage.
- **Machine learning** : le programme apprend ses paramètres à partir de données. Les modèles classiques (régression, arbres, boosting) restent des concurrents sérieux, surtout sur le tabulaire.
- **Deep learning** : des réseaux de neurones à plusieurs couches qui apprennent aussi leurs **représentations** (features). Même optimisation, même méthode pour toutes les architectures.
- **Transformers / LLM** : une architecture du deep learning (2017) devenue dominante sur le texte, puis les images et l'audio. Un LLM est un très grand Transformer entraîné à prédire le mot suivant.

Machine learning classique et deep learning : ce qui change vraiment

ML classique

- **Vous** concevez les features (ratios, agrégats, indicateurs).
- Le modèle apprend une décision sur ces features : régression, arbre, boosting, SVM.
- Excellent sur le **tabulaire**, interprétable, rapide, peu de données.
- Plafonne quand les features utiles sont impossibles à écrire (pixels, sons, mots).

Deep learning

- Le réseau apprend **les features et la décision** ensemble, couche après couche.
- Même recette pour tout : une architecture, une perte, une descente de gradient.
- Domine sur les données **non structurées** (image, audio, texte) et à grande échelle.
- Demande plus de données, de calcul et de méthode (J5) ; moins interprétable.

Ce qui ne change pas

- Découpage train / dev / test, sur-apprentissage, métriques, validation.
- La baseline classique reste obligatoire : c'est elle qu'il faut battre.
- PCA, ARIMA, filtres d'images : des outils du cercle 2 utilisés dans le cercle 3.
- Le bon sens métier : un modèle ne vaut que par la décision qu'il améliore.

Soixante ans en une frise : de quoi les Transformers sont-ils faits ?

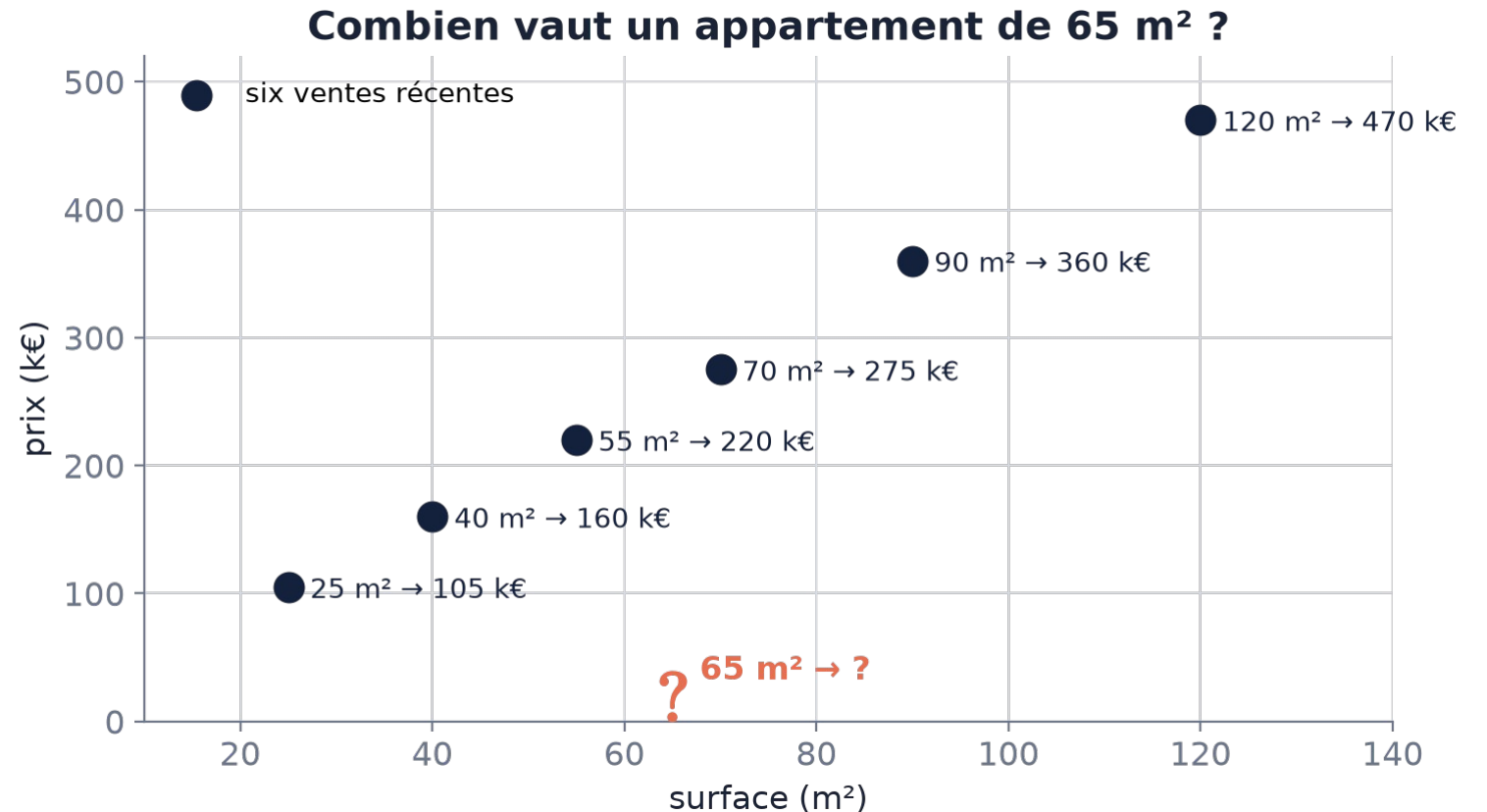
Chaque étape lève une limite de la précédente — c'est exactement le plan de ce cours



- **J1, J2, J5** : neurone, backpropagation, optimisation — les idées de 1958 à 1986, avec les outils de 2026.
- **J3, J6** : séries temporelles, réseaux récurrents, embeddings, attention — 1997 à 2017 ; **J4** : mettre tout cela en production.
- **J7–J8** : convolutions, ResNet, transfert, génératif — 1998 à 2015.
- **J8, J9** : soutenances, puis Transformers et LLM — 2017 à aujourd'hui.
- Rien n'est apparu d'un coup : chaque brique répond à une limite mesurable de la précédente (profondeur, mémoire, parallélisme). Retenir la **limite** vaut mieux que retenir la date.

Le problème du jour : six appartements vendus, combien vaut le septième ?

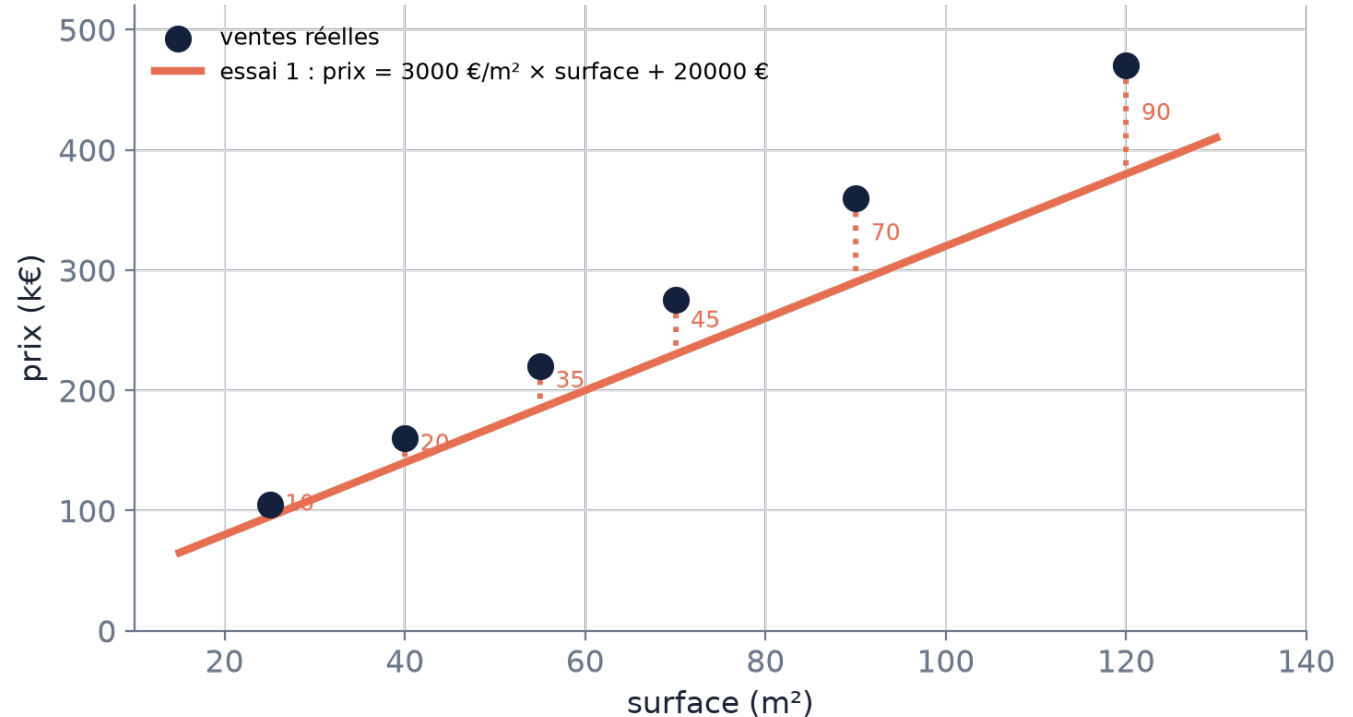
- Six ventes récentes : pour chacune, la **surface** et le **prix**.
- Un septième appartement, 65 m², arrive sur le marché. Quel prix annoncer ?
- Personne ne connaît la formule du prix. On va la **deviner à partir des exemples**.
- C'est exactement ce que fait un programme qui « apprend ».



Premier essai : une règle simple, $\text{prix} = w \times \text{surface} + b$

- On choisit la forme la plus simple : une **droite**. Deux nombres à fixer : **w**, le prix au m^2 , et **b**, une part fixe.
- Premier essai au jugé : $w = 3\,000 \text{ €/m}^2$, $b = 20\,000 \text{ €}$.
- Pour chaque vente, on mesure l'**écart** entre le prix prédit par la droite et le vrai prix (les pointillés).
- Écart moyen : **45 k€** par appartement. Ce nombre est notre **mesure d'erreur**.

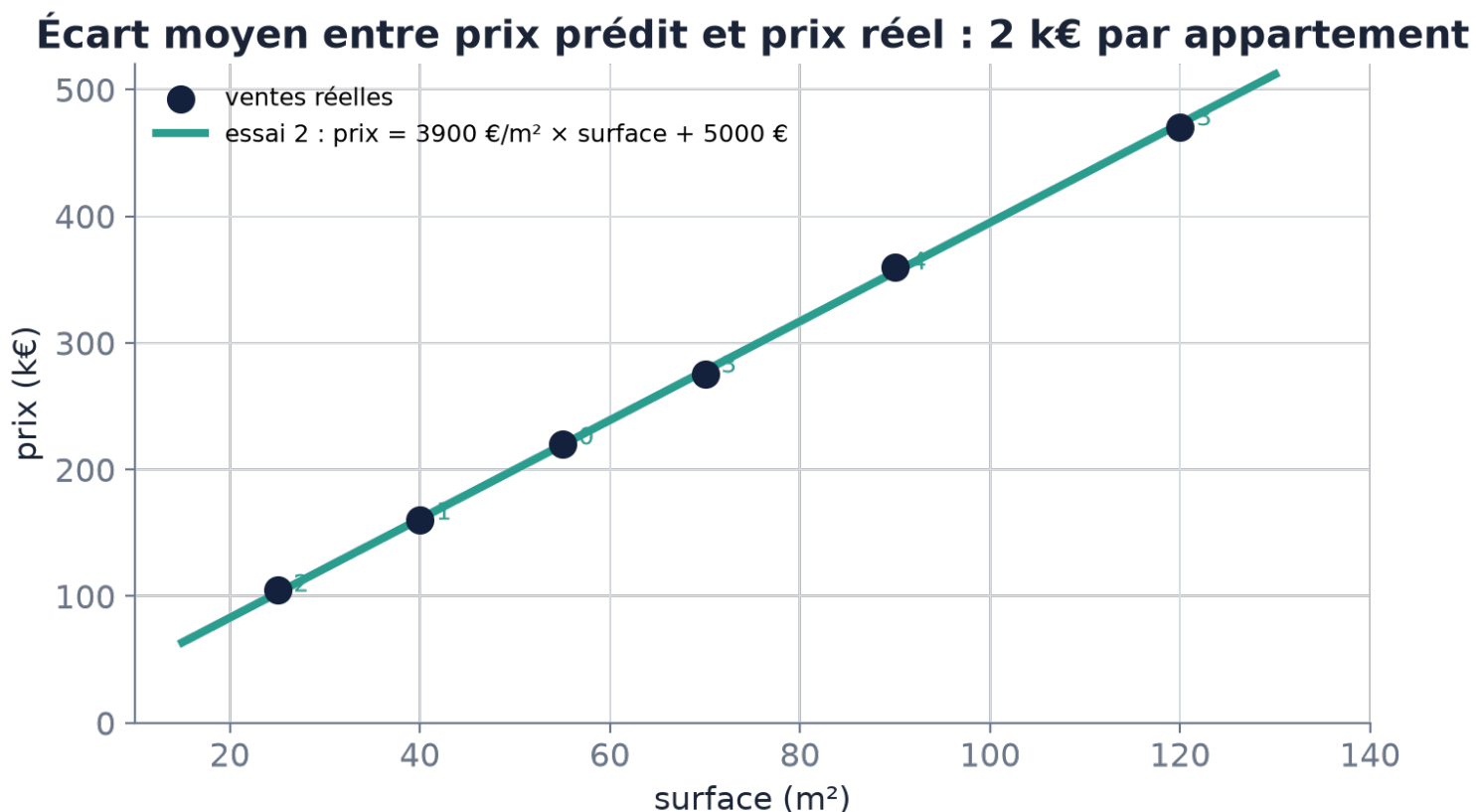
Écart moyen entre prix prédit et prix réel : 45 k€ par appartement



Remarque • b est la part du prix qui ne dépend pas de la surface (frais, parking, cave...). Sans b , la droite devrait passer par $(0, 0)$ et collerait mal aux données. On l'appellera le **biais** du neurone.

Deuxième essai : on change w et b, l'erreur baisse

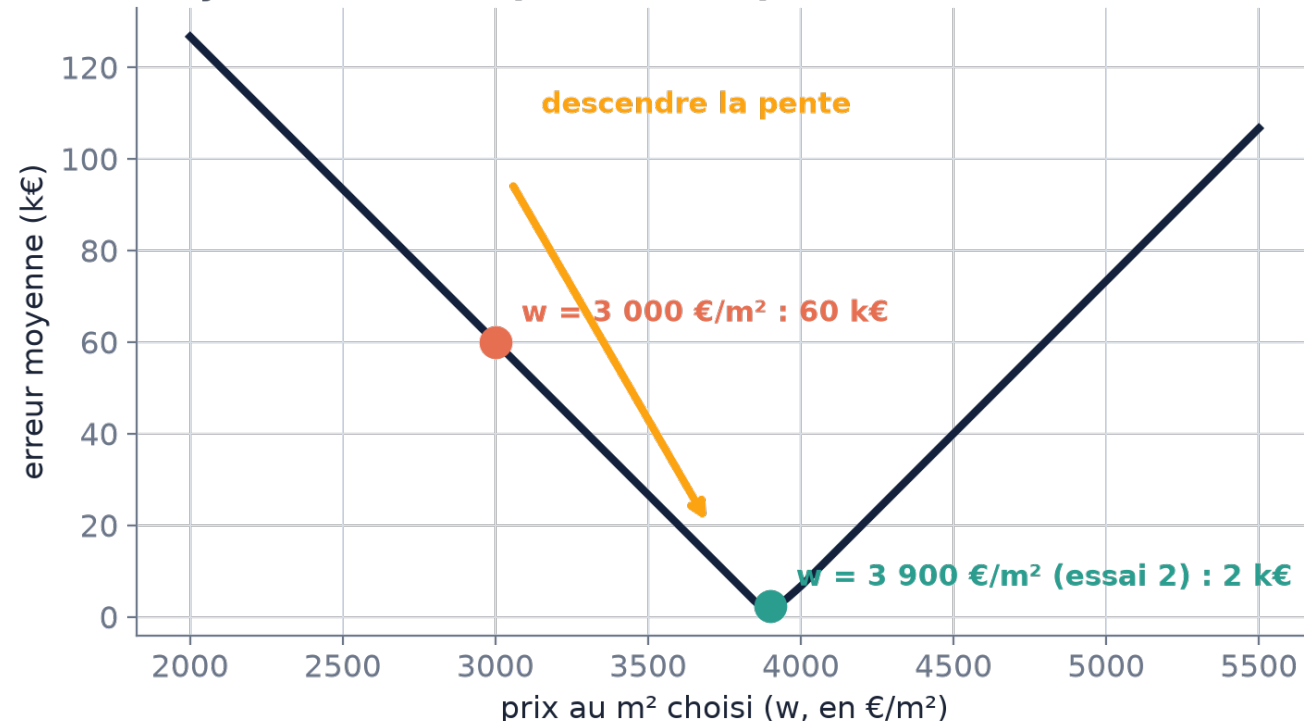
- Nouvel essai : $w = 3\,900 \text{ €/m}^2$, $b = 5\,000 \text{ €}$.
- Écart moyen : **2 k€**. Vingt fois mieux.
- **L'erreur est le juge** : pas besoin d'un expert, un seul nombre dit lequel des deux essais est le meilleur.
- Reste une question : comment trouver de bons w et b **sans deviner** ?



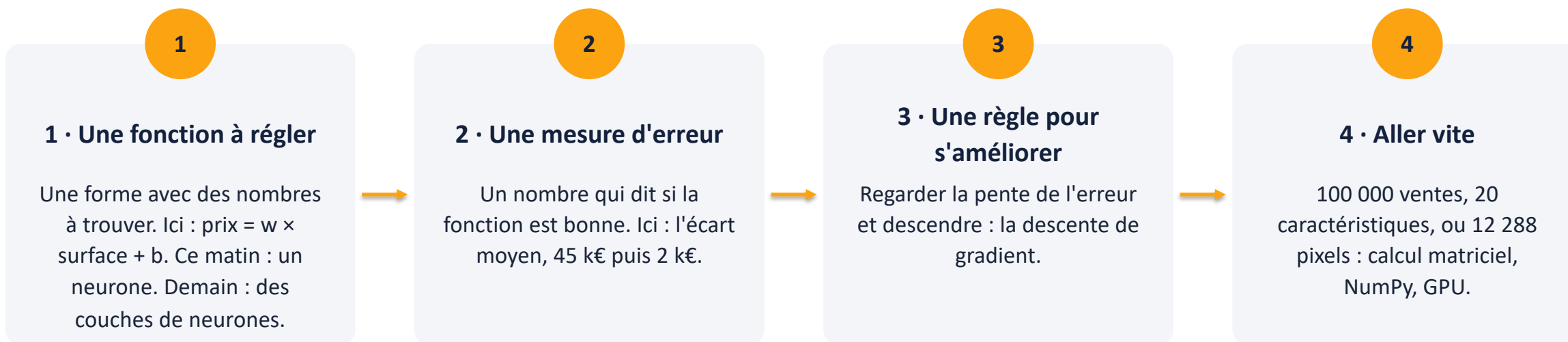
Améliorer sans deviner : regarder dans quel sens bouger

- Essai 1 : les prix prédits sont **trop bas**, surtout pour les grands appartements. Donc il faut **augmenter** w .
- Fixons b et traçons l'erreur pour chaque valeur de w : on obtient une **vallée**. Le meilleur w est au fond.
- Améliorer = **descendre la pente**, un petit pas à la fois. C'est la **descente de gradient**, le moteur de tout le deep learning.
- L'ordinateur refait ce geste des milliers de fois, pour des milliers de paramètres.

Erreur moyenne selon le prix au m^2 (part fixe $b = 5\,000\text{ €}$) : une vallée

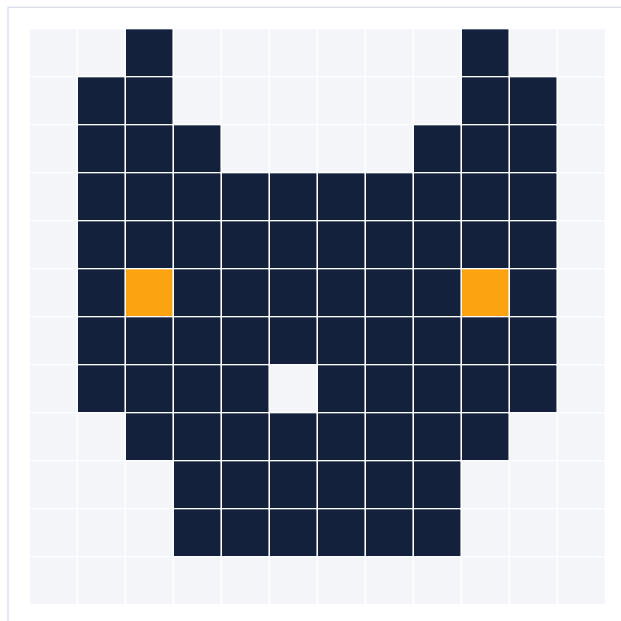


Les quatre briques de l'apprentissage



- Tout le deep learning tient dans ces quatre briques. Les huit journées suivantes ne changent presque que la brique 1 : la forme de la fonction.
- Quand une formule apparaît aujourd'hui, demandez-vous : « à quelle brique sert-elle ? ».

Pour l'ordinateur, une photo n'est qu'une grille de nombres



Ce que vous voyez

Une photo de 64 pixels de large sur 64 de haut (4 096 pixels), simplifiée ici en 12×12 .



12	14	11	15	13	12
18	22	19	24	21	20
200	214	208	219	205	210
231	228	235	224	233	229
226	240	219	233	237	221
15	236	21	230	230	18

Ce que voit l'ordinateur

Chaque pixel est un nombre de 0 (noir) à 255 (blanc). En couleur, trois nombres par pixel : sa part de rouge, de vert et de bleu.



$x_1 = 12$
 $x_2 = 14$
 $x_3 = 11$
 $\vdots$
 $x_{12\ 287} = 230$
 $x_{12\ 288} = 18$

12 288 nombres

64×64 pixels $\times$ 3 couleurs (rouge, vert, bleu) = 12 288. C'est l'entrée du modèle, comme la surface pour le prix.

- Pour l'appartement, l'entrée était **un seul nombre** (la surface). Pour la photo, ce sont **12 288 nombres**. Rien d'autre ne change.
- L'étiquette aussi est un nombre : **1** s'il y a un chat, **0** sinon. C'est le y que le modèle doit retrouver.

Le neurone fait un seul calcul : des pixels à une probabilité de chat



- Pour le prix, on avait **prix = w × surface + b**. Ici, la même somme pondérée, mais sur 12 288 pixels, puis un **écrasement entre 0 et 1** pour obtenir une probabilité.
- Au départ, w et b sont au hasard et le neurone se trompe souvent. Sur des milliers de photos, on répète : **calculer, comparer à l'étiquette, corriger**.

Et pour reconnaître un chat sur une photo ? La même recette

1 Les exemples

Des photos avec leur étiquette : chat ou pas chat.
Une photo 64×64 en couleur, c'est 12 288 nombres (les pixels).

2 La fonction

Un neurone : une combinaison des 12 288 pixels, écrasée entre 0 et 1 pour donner une **probabilité** de chat.

3 L'erreur

Un nombre qui pénalise une probabilité élevée quand il n'y a pas de chat, et inversement : la perte logistique (section 3).

4 S'améliorer

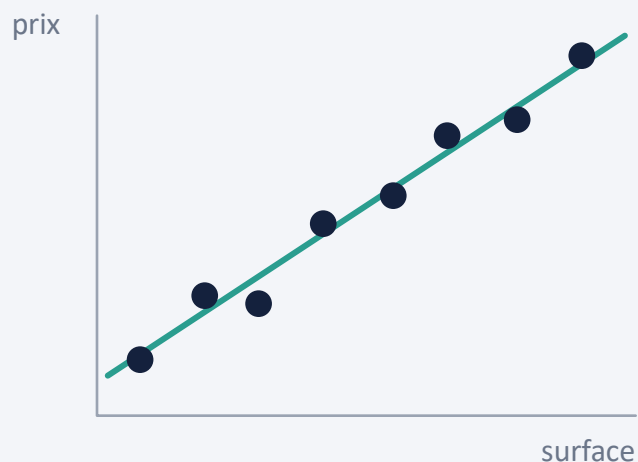
La même descente de pente, sur 12 289 nombres au lieu de deux.

	Prix d'un appartement	Chat sur une photo
Entrée x	1 nombre : la surface	12 288 nombres : les pixels
Sortie	un prix en k€	une probabilité entre 0 et 1
Paramètres	2 : w et b	12 289 : $w_1 \dots w_{12\,288}$ et b
Erreur	écart moyen en k€	perte logistique (section 3)

Une droite suffit pour un prix, pas pour répondre oui ou non

Prédire un prix

Une droite colle aux données



✓ ça marche

Répondre oui ou non

Une droite sort de 0 et 1 et hésite



✗ ça ne marche pas

Répondre oui ou non

Une droite pliée : 0 à gauche, 1 à droite



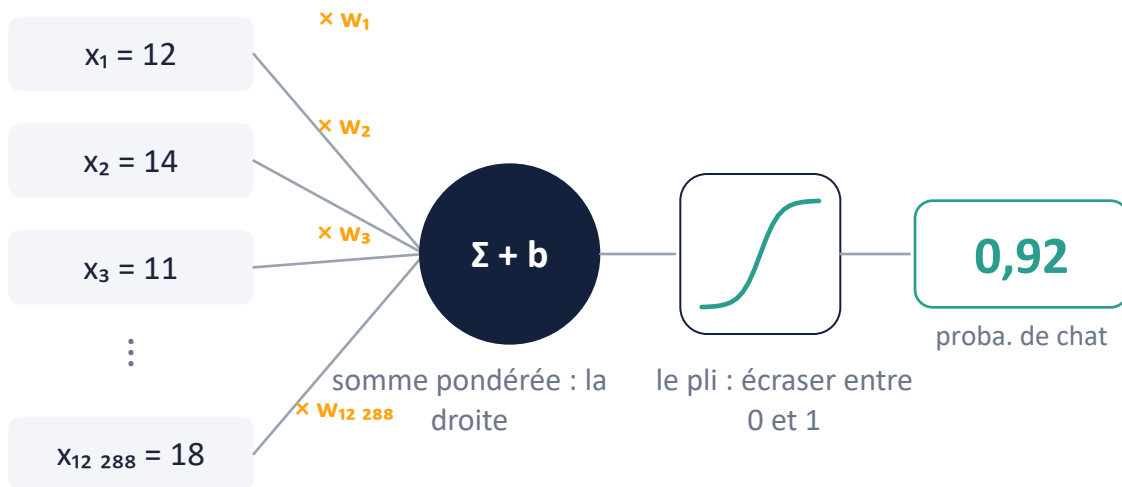
✓ ça marche

● exemples (pas chat = 0) ● exemples (chat = 1)

- Il faut donc une forme **un peu plus souple qu'une droite**, mais qu'on sache toujours **régler par petits pas** avec la descente de gradient.
- La droite pliée du panneau de droite, c'est exactement **un neurone** : une droite, puis un écrasement entre 0 et 1.

Un neurone = une droite linéaire + une courbe en S ; empilés, ils dessinent tout

UN NEURONE, DE L'INTÉRIEUR



$$p = \sigma(w_1x_1 + w_2x_2 + \dots + w_{12\ 288}x_{12\ 288} + b)$$

Les 12 289 nombres w et b sont ce que l'apprentissage règle. σ est fixé une fois pour toutes.

CE QUE DESSINENT 1, 3 PUIS 50 NEURONES



Chaque neurone ajoute un pli. En en empilant assez, on approche n'importe quelle forme : un prix, un contour, un mot.

CNN, LSTM, Transformers, GPT : le même neurone, arrangé autrement.

- Pourquoi pas un arbre de décision, souple lui aussi ? Parce qu'il ne se **règle pas par petits pas** : le neurone est la plus petite brique à la fois souple et réglable par descente de gradient.
- Une couche de neurones se calcule d'un coup en **matrices** : c'est pour ça que le GPU va si vite.

Pourquoi commencer par un neurone ?

Le problème à résoudre

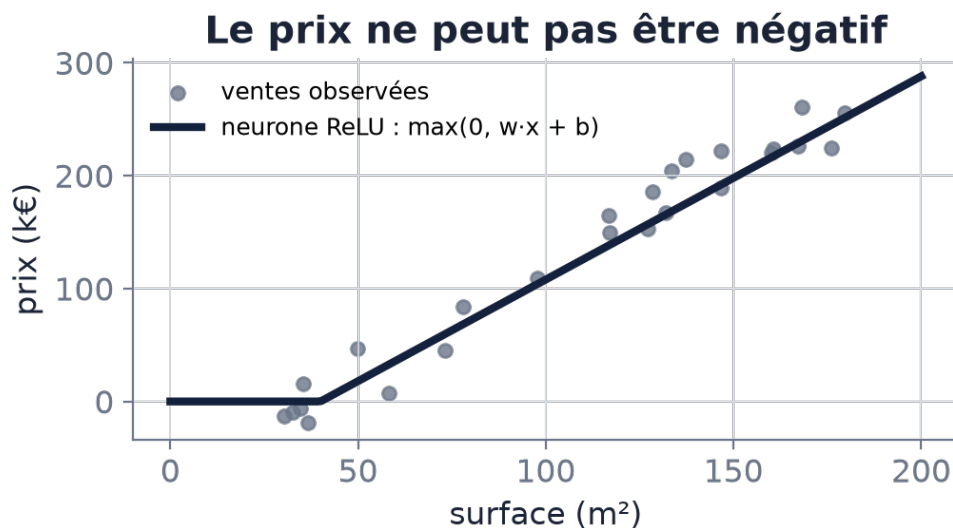
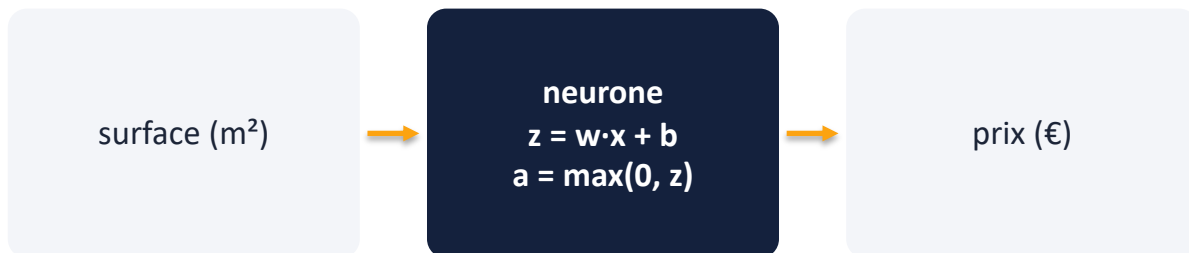
- Il faut une forme de fonction assez **souple** pour représenter un prix, une probabilité, un contour, un mot.
- Une droite seule est trop rigide : elle ne sait pas faire un seuil ni reconnaître un chat.
- Un arbre de décision est souple, mais on ne peut pas l'améliorer **par petits pas**.
- Le neurone est la plus petite brique à la fois **non linéaire** et **améliorable par petits pas**.

Pourquoi cette brique

- Un neurone seul : une droite pliée. Des neurones **empilés** : n'importe quelle fonction, ou presque.
- CNN, LSTM, Transformers, GPT : tous sont faits de ce même neurone, arrangé autrement.
- Une couche de neurones se calcule d'un coup, en matrices : d'où la vitesse sur GPU.
- Ce matin : un neurone, compris à fond. Demain : des milliers, sans rien de nouveau.

Un neurone : la plus petite fonction apprise

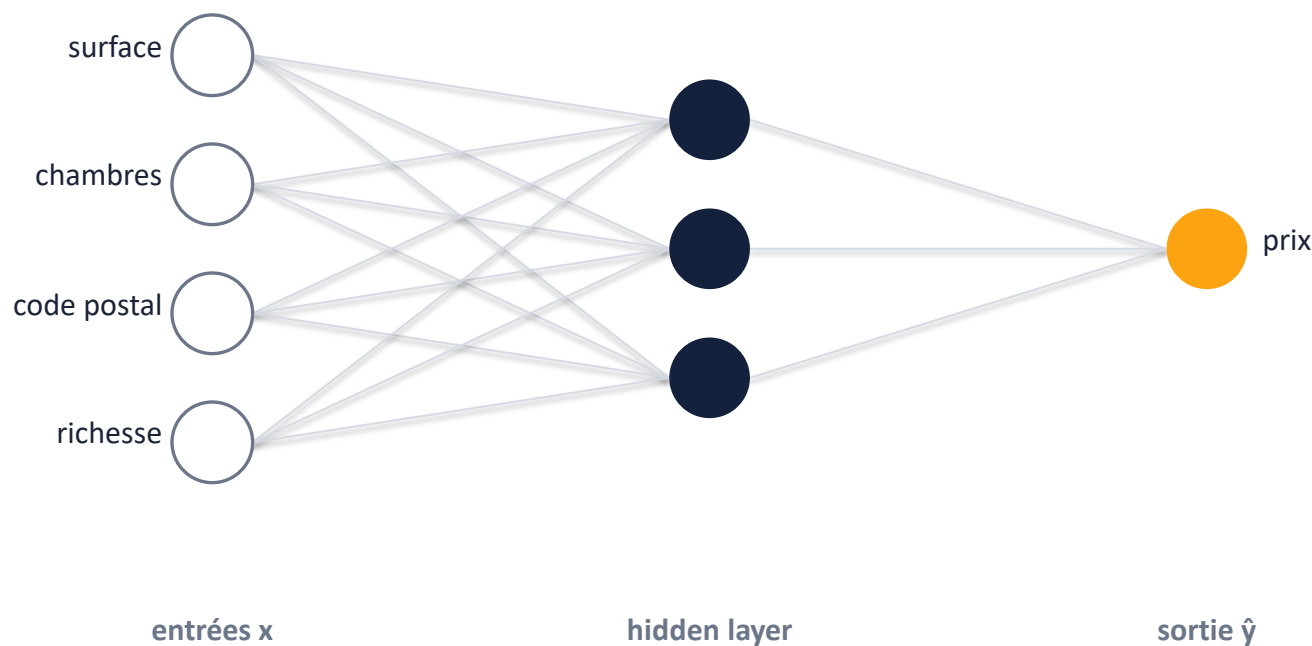
Exemple : prédire le prix d'une maison à partir de sa surface



- Un neurone calcule une **combinaison linéaire** ($z = w \cdot x + b$), puis applique une **fonction d'activation** à z .
- Ici l'activation est la **ReLU** (*Rectified Linear Unit*) : $a = \max(0, z)$.
 - z positif : elle le laisse passer tel quel. z négatif : elle renvoie 0.
 - Petite surface : la droite passerait sous zéro ; la ReLU bloque à 0, car **un prix ne peut pas être négatif** (le plat à gauche du graphique).
 - Activation la plus utilisée : simple, rapide, et c'est elle qui apporte le « pli ».
- **w** et **b** sont **appris** à partir d'exemples (surface, prix).
- Un réseau de neurones = beaucoup de ces briques, en couches.

Empiler des neurones : le réseau apprend ses propres features

Les couches intermédiaires ne sont pas définies à la main : elles émergent de l'entraînement



- Chaque unité du **hidden layer** est connectée à **toutes** les entrées : le réseau décide lui-même ce que représentent « taille de la famille », « accessibilité », « qualité de l'école »...
- On fournit seulement des paires **(x, y)** : la surface, les chambres, le code postal... et le prix observé.
 - Personne ne dit au réseau quoi calculer au milieu : il apprend seul les combinaisons d'entrées utiles pour prédire y.
 - Le réseau est dit **dense** (*fully connected*) : chaque neurone d'une couche reçoit toutes les sorties de la couche précédente.
- Plus de données $\Rightarrow$ plus de couches utiles $\Rightarrow$ « deep » learning.

L'apprentissage supervisé : presque toute la valeur économique du DL

Entrée x	Sortie y	Application	Architecture
Caractéristiques d'une maison	Prix	Immobilier	Réseau dense (MLP)
Publicité + profil utilisateur	Clic ? (0/1)	Publicité en ligne	Réseau dense
Image	Classe (1...1000)	Étiquetage de photos	CNN — journées 4 et 5
Audio	Transcription	Reconnaissance vocale	RNN / Transformer — journées 6 à 8
Français	Anglais	Traduction automatique	Seq2seq + attention — journée 6
Image + radar	Position des véhicules	Conduite autonome	Hybride / détection — journée 8

Données structurées vs non structurées

Structurées

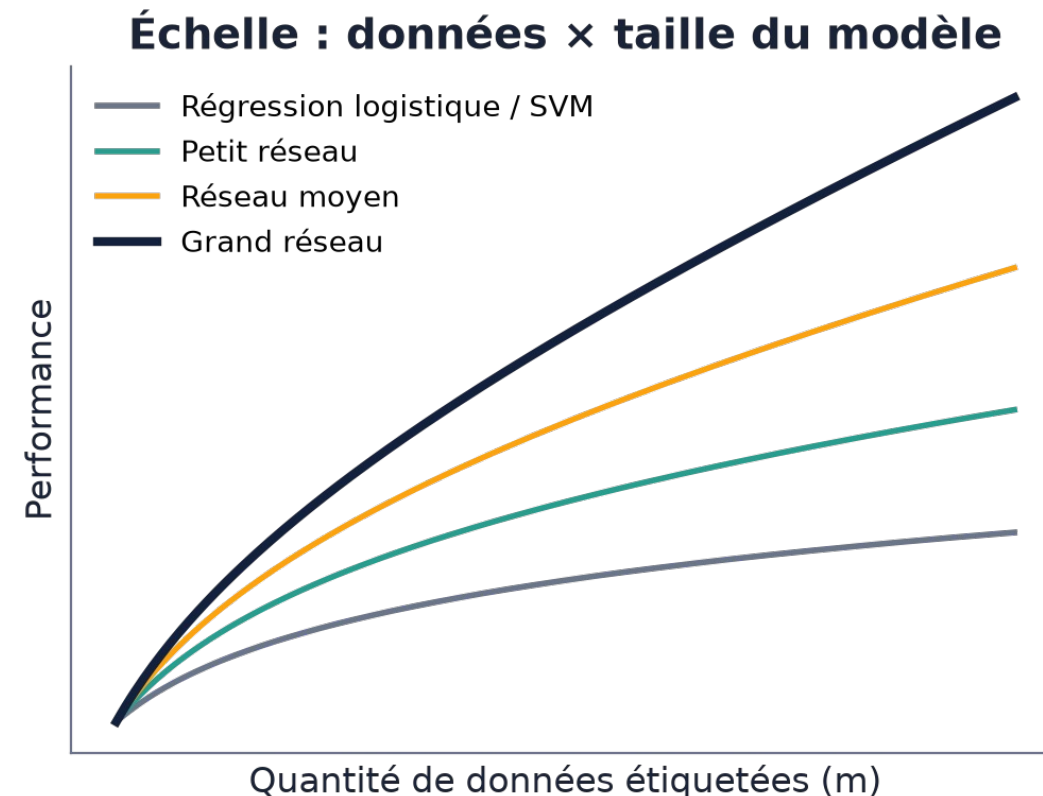
- Tableaux : colonnes nommées, valeurs numériques ou catégorielles.
- Exemples : transactions, logs, CRM, séries temporelles.
- Le gradient boosting reste souvent **très compétitif** — le DL n'est pas automatique ici (voir projet D3).
- Le DL apporte surtout sur les **séquences d'événements** et les **embeddings** de catégories.

Non structurées

- Images, audio, texte : pas de colonnes, signal brut.
- Historiquement **difficile pour les machines**, facile pour les humains.
- C'est là que le DL a changé la donne : CNN, RNN, Transformers.
- Les techniques de ce cours s'appliquent aux **deux** familles.

Pourquoi le deep learning décolle maintenant

- **Données** : numérisation, capteurs, web — les modèles classiques saturent, les grands réseaux continuent de progresser.
- **Calcul** : GPU, puis TPU ; un entraînement de plusieurs semaines en 2012 prend quelques heures aujourd'hui.
- **Algorithmes** : ReLU au lieu de sigmoïde (gradients qui ne s'écrasent plus), meilleures initialisations, BatchNorm, Adam, attention...
- Conséquence : la boucle **idée** → **code** → **expérience** tourne plus vite, donc on itère plus.
- Dans le régime « petites données », le classement des algorithmes dépend surtout du **feature engineering**.



Performance selon le volume de données, pour des modèles de taille croissante

La boucle d'itération : le vrai moteur du progrès



- Plus le cycle est court, plus on apprend vite : d'où l'importance de la **vectorisation** (aujourd'hui) et des **frameworks**
- La journée 5 (stratégie ML) est entièrement consacrée à rendre cette boucle efficace.

QUIZ KAHOOT · PARTIE 1 · NON NOTÉ

Un projet de machine learning, et pourquoi le deep learning

12 questions · 10 minutes · vers 10h40, avant la pause

1

Sortez votre téléphone

ou un second onglet

2

Allez sur kahoot.it

ou l'application Kahoot

3

Entrez le code PIN

affiché sur l'écran du quiz

On bascule sur le Kahoot en direct ; les questions et leurs corrections sont dans la fiche récap de ce soir.

3

Rappels mathématiques ciblés

Les outils des ingrédients 2, 3 et 4 : uniquement ce qui servira dans la journée

Quatre outils, pas plus



Le produit matriciel

À QUOI ÇA SERT

Appliquer $w \cdot x + b$ à des milliers d'exemples en **une seule opération**, pas en boucle : des minutes au lieu d'heures.

$X (1\ 000 \times 20) \cdot W (20 \times 1)$
= 1 000 prix en une ligne

Brique 4 · aller vite

détail slide 44



La dérivée

À QUOI ÇA SERT

Savoir dans quel sens bouger chaque nombre pour faire baisser l'erreur.

pente $> 0 \rightarrow$ diminuer w
pente $< 0 \rightarrow$ augmenter w

Brique 3 · s'améliorer

détail slide 45



La règle de la chaîne

À QUOI ÇA SERT

Remonter l'erreur à travers les couches jusqu'à chaque poids : la backpropagation.

$d \text{ erreur} / d w =$
 $d \text{ erreur} / d \text{ sortie} \times d$
 $\text{sortie} / d w$

Brique 3 · s'améliorer

détail slide 46



Moyenne et écart-type

À QUOI ÇA SERT

Mettre toutes les entrées à la même échelle avant d'apprendre, et résumer un jeu de données.

$x' = (x - \text{moyenne}) / \text{écart-type}$
 $\rightarrow$ centré sur 0, étalement 1

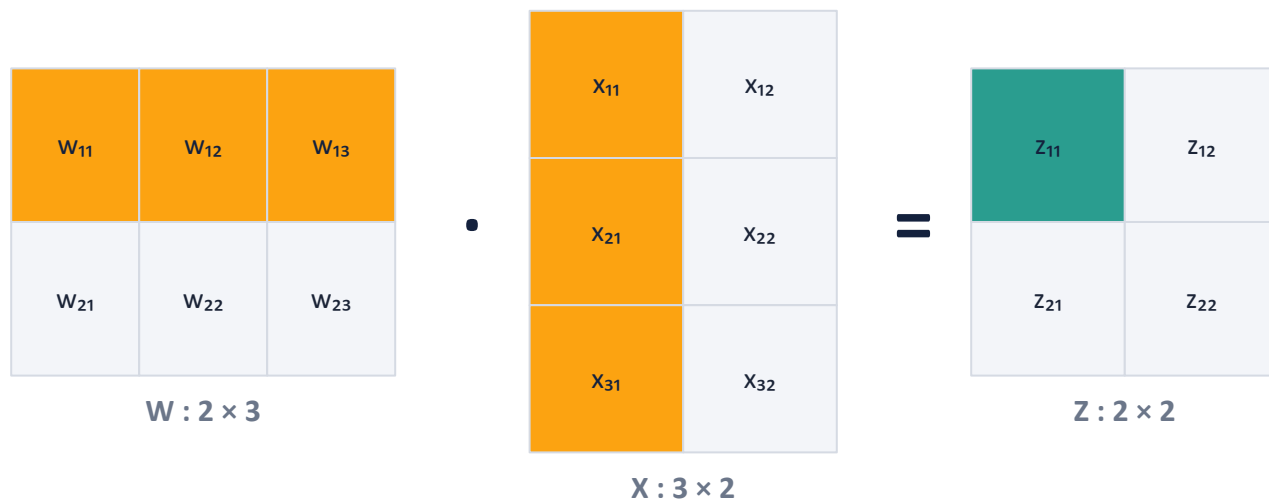
Brique 1 · la fonction

détail slide 47

Tout le reste des maths (probabilités, algèbre avancée) viendra quand on en aura besoin, jamais avant.

Produit matriciel : la règle des dimensions

$(n \times k) \cdot (k \times m) = (n \times m)$ — la dimension intérieure doit coïncider



- Ligne de $W \times$ colonne de $X \rightarrow$ une case de Z .
- **Transposée** : $(A^T)_{ij} = A_{ji}$; $(AB)^T = B^T A^T$.
- **Produit scalaire** : $w^T x = \sum w_k x_k$, un cas $(1 \times n)(n \times 1)$.
- **Hadamard** (élément par élément) : $A * B$, même forme. En NumPy : $A * B$ vs $A @ B$.
- Réflexe à prendre : écrire les dimensions **avant** de coder.

$$z_{11} = w_{11}x_{11} + w_{12}x_{21} + w_{13}x_{31} = \sum_k w_{1k} x_{k1}$$

POURQUOI ON EN A BESOIN

Un réseau applique la même formule $w \cdot x + b$ à chaque exemple. Avec 100 000 appartements et 50 neurones, ce sont **5 millions de petits calculs par passe**, et des milliers de passes. En boucle Python : des heures. En un produit de matrices $X \cdot W$: **une seule opération** que NumPy et le GPU exécutent en parallèle, en millisecondes. C'est la brique 4 : sans elle, pas de deep learning.

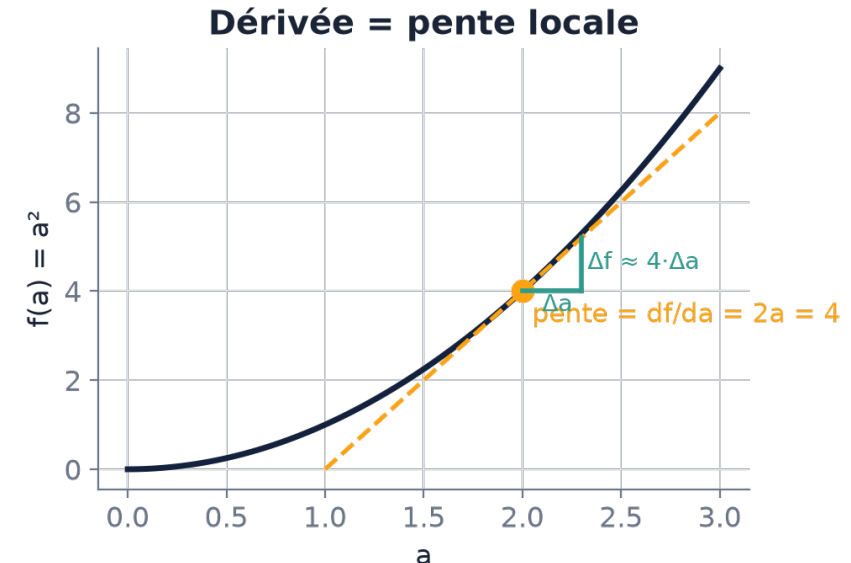
Dérivée = pente locale = « si je bouge a de ε , f bouge de $f'(a) \cdot \varepsilon$ »

$$\frac{df}{da}(a) = \lim_{\varepsilon \rightarrow 0} \frac{f(a + \varepsilon) - f(a)}{\varepsilon}$$

POURQUOI ON EN A BESOIN

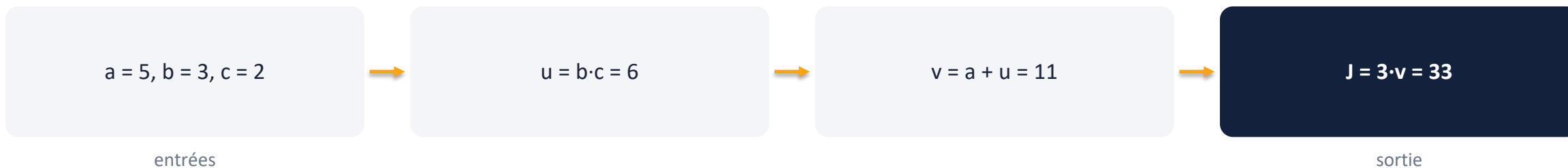
Notre détecteur de chats a **12 289 réglages** : un poids w par pixel (12 288) plus le biais b . Entraîner, c'est trouver la bonne valeur de chacun, et on ne peut pas les tester un par un. La dérivée répond pour chaque réglage : **si je l'augmente un peu, l'erreur monte ou descend ?** C'est tout ce dont la descente de gradient a besoin.

- $f(a) = a^2$ en $a = 2$: $f'(2) = 4$. Bouger a de 0,001 bouge f de $\approx 0,004$.
- La pente **dépend du point** : en $a = 5$, $f' = 10$.
- Dérivées à connaître : $(a^n)' = n \cdot a^{n-1}$; $(e^a)' = e^a$; $(\log a)' = 1/a$; $\sigma' = \sigma(1-\sigma)$.
- **Gradient** $\nabla J(w)$: le vecteur des dérivées partielles $\partial J / \partial w_k$ — la direction de plus forte montée.



Règle de la chaîne : le cœur de la backpropagation

Si J dépend de v qui dépend de a , alors $dJ/da = (dJ/dv) \cdot (dv/da)$



$$\frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da} = 3 \cdot 1 = 3$$

$$\frac{dJ}{db} = \frac{dJ}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{db} = 3 \cdot 1 \cdot c = 6$$

$$\frac{dJ}{dc} = 3 \cdot 1 \cdot b = 9$$

- **Passe avant** (gauche → droite) : calculer J . **Passe arrière** (droite → gauche) : propager $dJ/d(\cdot)$ en multipliant les dérivées locales.
- Convention de code : da désigne dJ/da . On calcule d'abord dv , puis du , da , db , dc — dans l'ordre inverse du graphe.

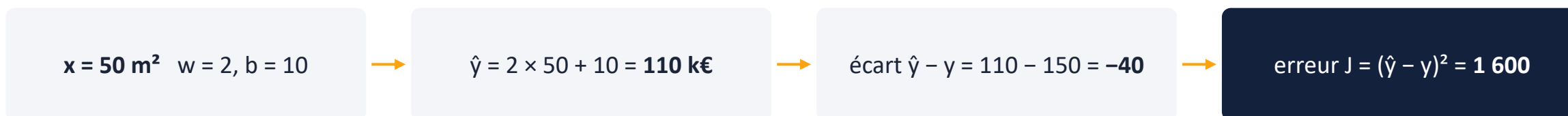
POURQUOI ON EN A BESOIN

Un poids de la première couche n'agit sur l'erreur qu'à **travers toutes les couches suivantes**. La règle de la chaîne calcule son effet en multipliant les effets locaux, de la sortie vers l'entrée : **une seule passe arrière donne la dérivée de tous les poids**. Sans elle, il faudrait recalculer tout le réseau pour chacun des 12 289 poids.

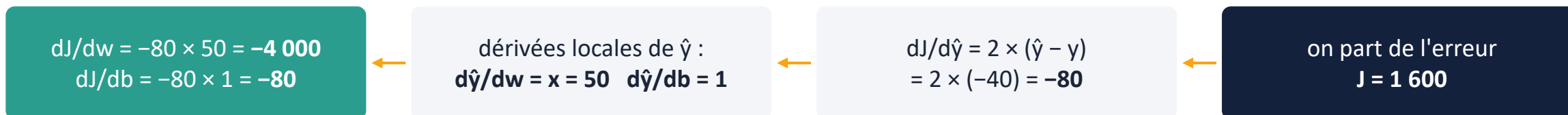
La backpropagation sur un exemple : un appartement

Surface 50 m², prix réel $y = 150$ k€. Le neurone prédit $\hat{y} = w \times \text{surface} + b$, avec au départ $w = 2$ et $b = 10$.

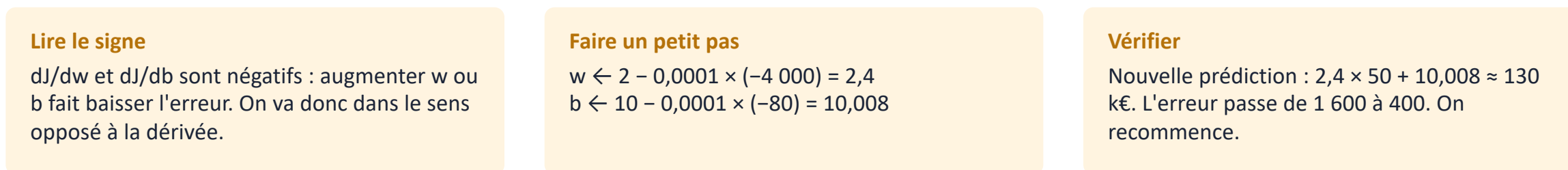
PASSE AVANT (gauche → droite) : calculer la prédiction et l'erreur



PASSE ARRIÈRE (droite → gauche) : remonter l'erreur jusqu'à w et b



LECTURE ET MISE À JOUR ($\alpha = 0,0001$)



- Avec 12 289 poids et des dizaines de couches, c'est **exactement le même calcul** : une passe avant, une passe arrière qui multiplie les dérivées locales de droite à gauche, puis un petit pas sur chaque poids. C'est ce que fait `loss.backward()` en une ligne.

Perte, gradient, backpropagation : trois rôles différents

La descente de gradient a besoin de la pente dJ/dw pour chaque poids. Elle ne dit pas comment l'obtenir : c'est le travail de la backpropagation.

MESURE

La loss function J

- Résume en un seul nombre à quel point on se trompe.
- Exemple : $J = 1\,600$ pour notre appartement.
- Ne dit pas quel poids bouger, ni dans quel sens.



DÉCIDE

La descente de gradient

- La règle : $w \leftarrow w - \alpha \cdot dJ/dw$, pour chaque poids.
- Suppose la pente dJ/dw déjà connue.
- Avec 12 289 poids, il faut 12 289 pentes à chaque pas.



CALCULE

La backpropagation

- Calcule toutes les pentes dJ/dw en une passe arrière.
- Outil : la règle de la chaîne (slide 46).
- Exemple : $dJ/dw = -4\,000$, $dJ/db = -80$.

POURQUOI ON NE PEUT PAS S'EN PASSER

Sans backpropagation : pour connaître la pente d'un poids, il faut le bouger un peu, refaire toute la passe avant et comparer. Soit 12 289 passes avant pour une seule mise à jour, à répéter des milliers de fois.

Avec backpropagation : une passe avant, une passe arrière, et les 12 289 pentes sont toutes là. Le coût d'une mise à jour est à peine celui de deux prédictions.

En une phrase : la perte dit **combien** on se trompe, la descente de gradient dit **quoi faire**, la backpropagation **calcule la pente**.

Moyenne et écart-type : mettre toutes les entrées à la même échelle

$x' = (x - \text{moyenne}) / \text{écart-type}$: chaque colonne devient centrée sur 0 avec un étalement de 1

AVANT : des échelles incomparables

	surface (m ²)	chambres	code postal
appt A	32	1	33000
appt B	85	3	33800
appt C	140	5	33100
moyenne	86	3	33300
écart-type	44	1,6	350

Un pas de 1 sur « chambres » et un pas de 1 sur « code postal » n'ont rien à voir, mais le même w les traite pareil : la descente zigzag.

APRÈS : tout entre -2 et +2

	surface'	chambres'	code postal'
appt A	-1,2	-1,2	-0,9
appt B	0,0	0,0	1,4
appt C	1,2	1,2	-0,6
moyenne	0	0	0
écart-type	1	1	1

Exemple : appt A, $\text{surface}' = (32 - 86) / 44 = -1,2$. Les trois colonnes parlent la même langue : un pas de 1 = un écart-type.

- **Moyenne** : la valeur « typique » d'une colonne. **Écart-type** : de combien les valeurs s'en éloignent en général. Deux nombres suffisent à résumer une colonne.
- **Standardiser** = soustraire la moyenne, diviser par l'écart-type. On le fait sur **chaque entrée x**, jamais sur la sortie y qu'on veut prédire.
- Les moyennes et écarts-types sont calculés **sur les données d'entraînement seulement**, puis réutilisés tels quels sur les nouvelles données (journée 5).
- Dans le code : StandardScaler de scikit-learn, ou deux lignes NumPy : $(X - X.\text{mean}(\theta)) / X.\text{std}(\theta)$.

4

La régression logistique vue comme un neurone

Ingrédients 1, 2 et 3 : une famille de fonctions, une mesure d'erreur, une règle pour s'améliorer

Classification binaire : de l'image au vecteur

Exemple canonique : cette image contient-elle un chat ? $y \in \{0, 1\}$



$$n_x = 64 \times 64 \times 3 = 12\,288 \quad X \in \mathbb{R}^{12288 \times m}, \quad Y \in \mathbb{R}^{1 \times m}$$

- Le neurone ne sait lire qu'une **liste de nombres** : il faut donc transformer la photo en une colonne de 12 288 valeurs. Les trois couches rouge, vert et bleu sont mises bout à bout dans ce seul vecteur x .
- Chaque photo donne une colonne ; m photos côte à côte forment la matrice X (12 288 lignes, m colonnes). Y est la ligne des réponses : 1 chat, 0 pas chat.
- En NumPy : `X = imgs.reshape(m, -1).T` → forme (12288, m). Vérifier `X.shape` **systématiquement**.
- Les pixels vont de 0 à 255 : on divise par 255 pour tout ramener entre 0 et 1 (standardisation simple). Sinon la descente de gradient saute (slide 47).

Une photo, une colonne : construire X et Y

m photos aplaties côte à côte forment X ($12\ 288 \times m$) ; Y ($1 \times m$) porte les réponses.

Chaque photo : $64 \times 64 \times 3 = 12\ 288$ nombres



Comment lire le schéma

Une photo = une colonne

On aplatit chaque image (slide 51) en une colonne de 12 288 nombres, un par pixel et par couleur.

m photos côte à côte = la matrice X

X a 12 288 lignes et m colonnes. Avec 1 000 photos, X fait $12\ 288 \times 1\ 000$: lire la ligne i , c'est voir le même pixel sur toutes les photos.

Y = la ligne des réponses

Une seule ligne, m colonnes : la colonne j de Y est la réponse pour la colonne j de X (1 chat, 0 pas chat).

Pourquoi cette disposition ?

Le neurone calcule $Z = w^T X + b$ en **une seule multiplication** pour les m photos (slide 44). Sans boucle Python.

Deuxième question : cet appartement se vendra-t-il en moins d'un mois ?

Ce qui change

- La réponse n'est plus un prix mais **oui ou non**. Ce qu'on veut : une **probabilité** entre 0 et 1.
- La droite $w \times x + b$ peut valoir -3 ou 12 : il faut l'**écraser** entre 0 et 1.
- C'est ce que fait la **sigmoïde** : une droite écrasée en forme de S. Un neurone, c'est une droite suivie d'une fonction de ce genre.

Ce qui ne change pas

- Même recette qu'avant : des exemples (appartement, vendu vite ou non), une fonction (droite + sigmoïde), une erreur, une descente de pente.
- Seule l'**erreur** change : on ne mesure plus un écart en k€ mais à quel point la probabilité annoncée était fausse.
- Et le chat ? Identique, avec 12 288 entrées au lieu d'une.

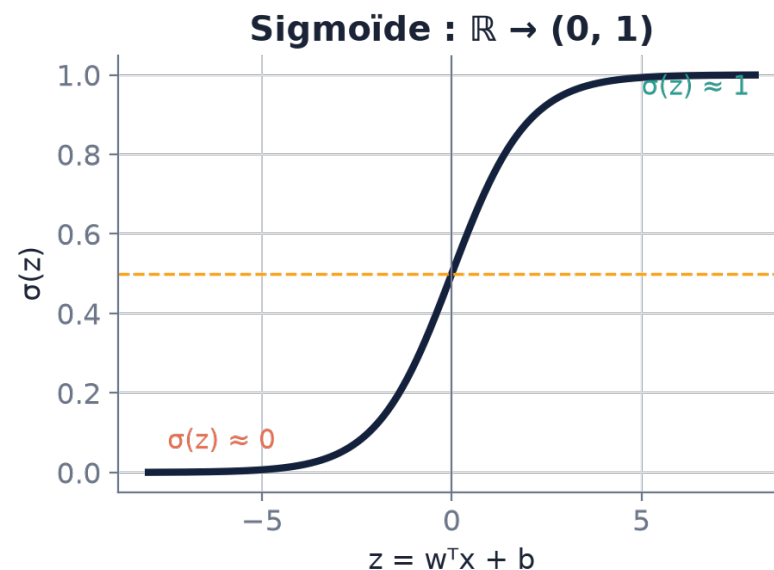
Régression logistique : un neurone à activation sigmoïde

Le même neurone qu'à la slide 34 : une droite $w^T x + b$, puis la courbe en S σ qui ramène le résultat entre 0 et 1 pour en faire une probabilité.

$$\hat{y} = \sigma(w^T x + b),$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Pourquoi ce nom ? **Régression** : on calcule une valeur à partir de x .
Logistique : cette valeur est une probabilité $P(y = 1 \mid x)$, donc entre 0 et 1, grâce à la **sigmoïde** σ . Réglages : w (un poids par entrée) et b .
- **Cas concret** : un client va-t-il résilier ? x = (ancienneté, réclamations, impayés...) ; $\hat{y} = 0,82$ signifie « 82 % de risque ». Pour le chat : $x = 12\,288$ pixels, $\hat{y}$ = probabilité d'un chat.
- Pourquoi pas $w^T x + b$ tout seul ? Une droite sort de $(0, 1)$; σ la ramène dedans : $\sigma(0) = 0,5$, $\sigma \rightarrow 1$ à droite, $\sigma \rightarrow 0$ à gauche.
- Décision : $\hat{y} \geq 0,5 \Leftrightarrow z \geq 0$: la **frontière est linéaire** en x (un hyperplan).



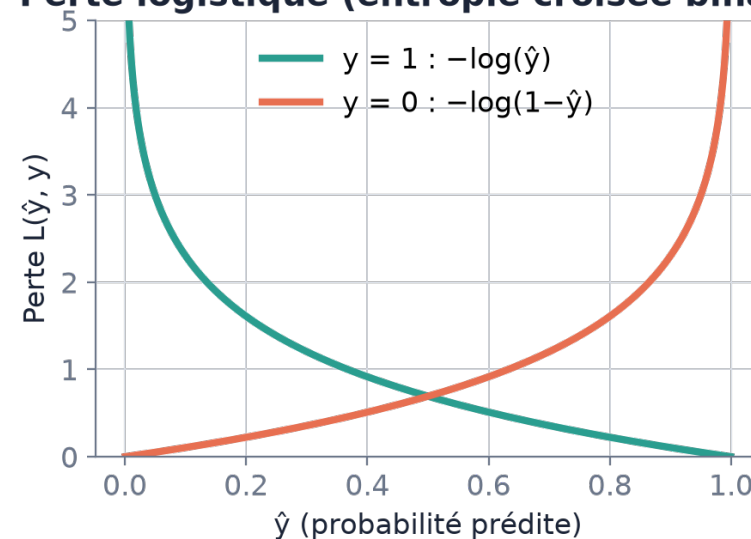
Perte et coût : pourquoi pas l'erreur quadratique ?

En clair : la perte mesure à quel point une prédiction est fautive. On la choisit en forme de bol (convexe) pour que la descente de gradient trouve toujours le fond.

$$\mathcal{L}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})], \quad J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- Avec $\frac{1}{2}(\hat{y}-y)^2$ le problème devient **non convexe** : plusieurs minima locaux, descente de gradient piégée.
- La perte logistique est **convexe** en (w, b) et découle du maximum de vraisemblance (rappel probas).
- **Perte $\mathcal{L}$** : un exemple. **Coût J** : moyenne sur le jeu d'entraînement. On minimise J .
- Si $y = 1$: on veut $\hat{y}$ grand $\rightarrow -\log \hat{y}$ petit. Si $y = 0$: on veut $\hat{y}$ petit $\rightarrow -\log(1-\hat{y})$ petit.
- **Cas concret** : un client a résilié ($y = 1$). Le modèle avait prédit 0,9 $\rightarrow$ perte 0,105 (bien). Il avait prédit 0,1 $\rightarrow$ perte 2,3 (très mal : confiant et faux). Sur 10 000 clients, le coût J est la moyenne de ces pertes : le nombre unique que l'on veut faire baisser.

Perte logistique (entropie croisée binaire)



Pourquoi la descente de gradient est le cœur de tout le deep learning

12 289

paramètres pour notre classifieur de chats (12 288 poids + 1 biais). Aucune formule fermée ne donne les meilleurs

10^{12}

paramètres dans un grand modèle de langage. Une recherche par grille sur 2 valeurs par paramètre demanderait $2^{(10^{12})}$ essais

1

algorithme pour les entraîner tous : suivre la pente du coût, par petits pas. De la régression logistique à GPT, c'est le même

EN CLAIR

Il y a trop de réglages pour les deviner ou les calculer d'un coup. On regarde seulement, pour chacun, **dans quel sens le bouger un peu pour que l'erreur baisse**, et on répète des milliers de fois. C'est la descente de gradient, et elle sert partout.

On ne peut ni résoudre l'équation $\partial J / \partial w = 0$ à la main (trop de paramètres, non linéaire), ni essayer toutes les valeurs. La seule information locale disponible est la pente : le gradient dit « si tu augmentes w_1 de ϵ , le coût varie de tant ». Descendre cette pente, un peu, des milliers de fois, est ce que l'on appelle « entraîner ». Tout ce que JS ajoutera (momentum, Adam) ne fait qu'accélérer cette descente.

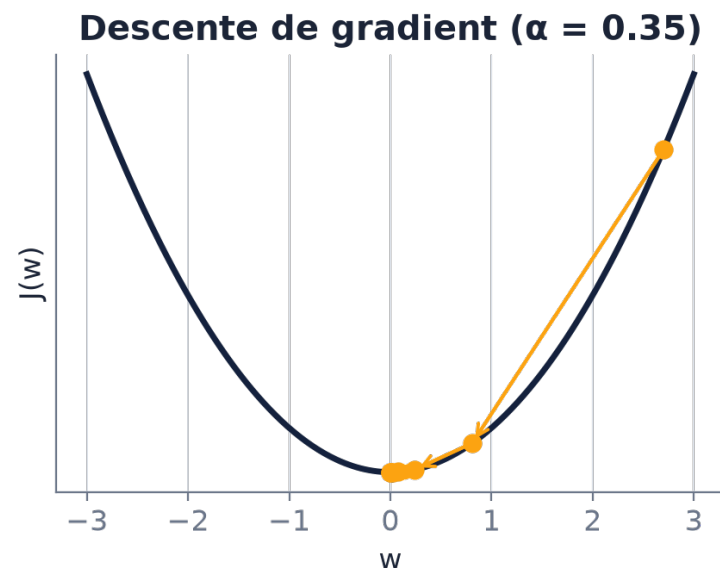
Descente de gradient : descendre la pente pas à pas

En clair : à chaque tour, on bouge chaque réglage d'un petit pas dans le sens qui fait baisser l'erreur ; α fixe la taille du pas.

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w},$$

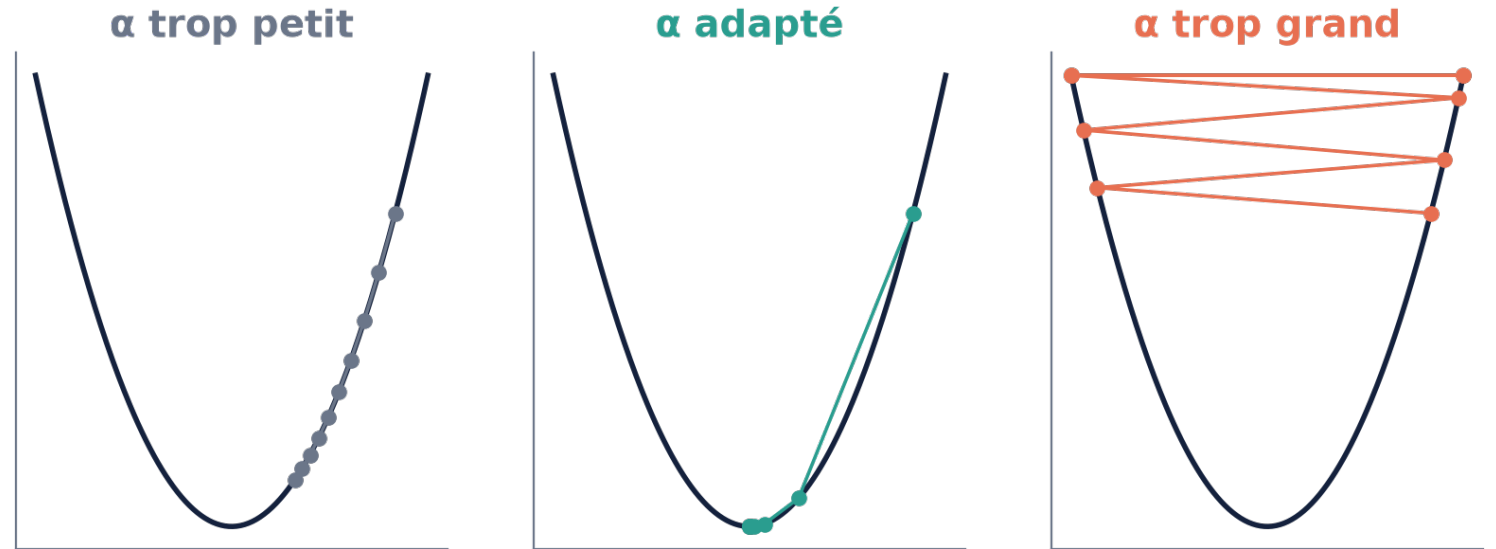
$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

- α : taux d'apprentissage (*learning rate*), le premier hyperparamètre à régler.
- **Cas concret** : reprenons les appartements. Si le prix au m² w est trop élevé, les prédictions dépassent les prix réels, la dérivée $\partial J / \partial w$ est positive : la règle **baisse** w . Si w est trop bas, la dérivée est négative : la règle **monte** w . Le gradient encode « dans quel sens corriger, et de combien ».
- Le gradient pointe vers la montée : on va dans le sens **opposé**.
- Convexité $\Rightarrow$ un seul minimum : n'importe quelle initialisation converge (pour la régression logistique seulement !).
- Notation de code : $dw = \partial J / \partial w$, $db = \partial J / \partial b$. Même forme que w et b .



Le taux d'apprentissage α : ni trop petit, ni trop grand

- Symptôme d'un α trop grand : le coût **augmente** ou oscille.
- Symptôme d'un α trop petit : le coût descend **très lentement**.
- Ordre de grandeur de départ : 10^{-3} à 10^{-1} selon la normalisation des données.
- On verra J5 : décroissance de α , Adam, recherche sur échelle log.



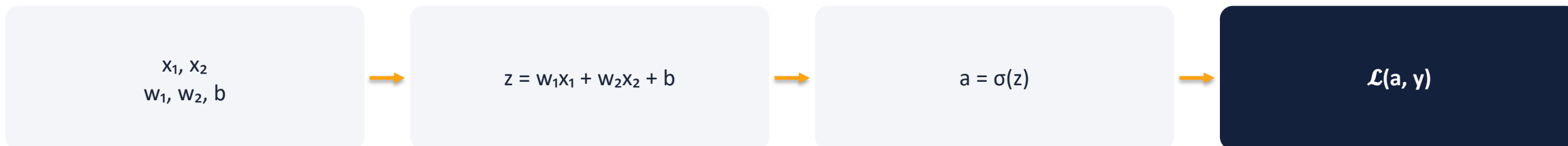
Trop petit : lent. Adapté : converge. Trop grand : diverge (oscille et s'éloigne).

EN CLAIR

α est la **taille du pas**. Trop petit : on met des heures à descendre. Trop grand : on saute par-dessus le fond du bol et on remonte de l'autre côté. On le règle en regardant la courbe du coût au fil des itérations.

Graphe de calcul de la régression logistique

Un exemple, deux features : forward puis backward



Passé arrière (dérivées de $\mathcal{L}$ par rapport à chaque nœud, de droite à gauche) :

$$da = \frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

dérivée de la perte

$$dz = da \cdot \frac{\partial a}{\partial z} = da \cdot a(1-a) = a - y$$

se simplifie remarquablement

$$dw_1 = x_1 dz, \quad dw_2 = x_2 dz, \quad db = dz$$

dérivées locales de z

EN CLAIR

On lit le graphe de droite à gauche : de combien l'erreur change si a bouge, puis si z bouge, puis si w_1, w_2, b bougent. Tout se résume à $\mathbf{dz} = \mathbf{a} - \mathbf{y}$: l'écart entre la prédiction et la vérité. Plus on se trompe, plus on corrige.

Descente de gradient sur m exemples : version « boucles » (à ne jamais écrire en vrai)

python

```
J = 0; dw1 = 0; dw2 = 0; db = 0
for i in range(m):                                # boucle sur les exemples
    z_i = w1 * x1[i] + w2 * x2[i] + b
    a_i = sigmoid(z_i)
    J += -(y[i] * log(a_i) + (1 - y[i]) * log(1 - a_i))
    dz_i = a_i - y[i]
    dw1 += x1[i] * dz_i                            # une 2e boucle cachée sur les features
    dw2 += x2[i] * dz_i
    db += dz_i
J /= m; dw1 /= m; dw2 /= m; db /= m
w1 -= alpha * dw1; w2 -= alpha * dw2; b -= alpha * db
```

- Deux boucles imbriquées : sur les **m exemples** et sur les **n_x features**.
- Sur 12 288 features et 10 000 images : des centaines de millions d'opérations Python interprétées.
- La vectorisation (cet après-midi) supprime **les deux** boucles.
- Un pas complet de descente de gradient = une passe sur tout le jeu (une **epoch**).

QUIZ KAHOOT · PARTIE 2 · NON NOTÉ

Maths, régression logistique, vectorisation et ACP

13 questions · 10 minutes · en fin de matinée, après la régression logistique

1

Sortez votre téléphone

ou un second onglet

2

Allez sur kahoot.it

ou l'application Kahoot

3

Entrez le code PIN

affiché sur l'écran du quiz

On bascule sur le Kahoot en direct ; les questions et leurs corrections sont dans la fiche récap de ce soir.

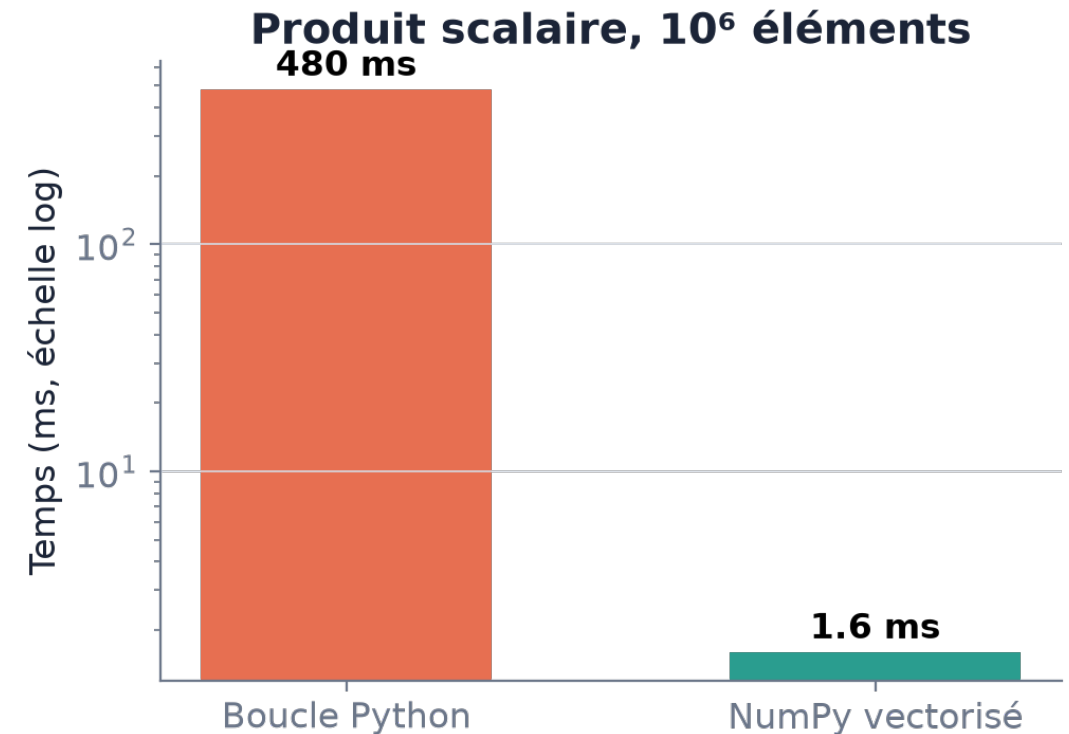
5

Vectorisation et NumPy

Ingrédient 4 : aller vite — supprimer les boucles, penser en matrices, éviter les pièges

Pourquoi vectoriser : SIMD, BLAS et GPU

- `np.dot(w, x)` appelle une routine **BLAS** compilée qui exploite le parallélisme du processeur (SIMD) ; la boucle Python interprète chaque opération.
- Facteur typique : **×100 à ×300** sur CPU. Sur GPU, des milliers de cœurs : encore plus.
- Règle : **dès qu'on écrit un `for` sur des nombres, chercher l'équivalent NumPy** (`np.dot`, `np.exp`, `np.sum`, `np.maximum`...).
- Les frameworks (PyTorch, TensorFlow) reposent sur le même principe : des tenseurs et des opérations vectorisées.



Mesure indicative : produit scalaire sur 10^6 éléments (à refaire en TP)

Vectoriser, en clair : un calcul à la fois ou des milliers d'un coup

SIMD : une instruction, plusieurs données

Le processeur applique la même opération à 8 ou 16 nombres d'un coup, comme un caissier qui scanne tout un plateau au lieu d'un article à la fois. La boucle Python passe les articles un par un.

BLAS : des routines écrites une fois pour toutes

Produit scalaire, produit matriciel... écrits en C par des spécialistes depuis 40 ans et réglés pour chaque processeur. np.dot ne calcule pas lui-même : il appelle ces routines.

GPU : des milliers de petits cœurs

Conçu pour afficher des millions de pixels en parallèle, il fait exactement ce dont un réseau a besoin : la même multiplication répétée des millions de fois. PyTorch lui envoie $X \cdot W$.

Ordre de grandeur sur notre détecteur de chats

$12\,288 \text{ pixels} \times 1\,000 \text{ photos} \times 1\,000 \text{ itérations} \approx 12 \text{ milliards}$ de multiplications à faire.

Boucle Python

un nombre à la fois, interprété



$\approx 3 \text{ h}$

NumPy sur CPU

SIMD + BLAS : $\times 100$ à $\times 300$



$\approx 1 \text{ min}$

PyTorch sur GPU

des milliers de cœurs



$\approx 1 \text{ s}$

Ordres de grandeur indicatifs ; les barres ne sont pas à l'échelle (vrai rapport : 1 à 10 000).

À RETENIR

Même formule, même résultat : seule la façon de l'exécuter change. C'est pour cela qu'on écrit tout en matrices.

Régression logistique entièrement vectorisée

Passe avant

b scalaire diffusé sur m colonnes

$$Z = w^T X + b \in \mathbb{R}^{1 \times m}, \quad A = \sigma(Z)$$

Passe arrière (1)

$$dZ = A - Y \in \mathbb{R}^{1 \times m}$$

m dérivées d'un coup

Passe arrière (2)

$$dw = \frac{1}{m} X dZ^T \in \mathbb{R}^{n_x \times 1}, \quad db = \frac{1}{m} \sum_i dZ^{(i)}$$

zéro boucle

Mise à jour

$$w \leftarrow w - \alpha dw, \quad b \leftarrow b - \alpha db$$

une itération = une epoch

La même chose en NumPy : 12 lignes

python

```
import numpy as np

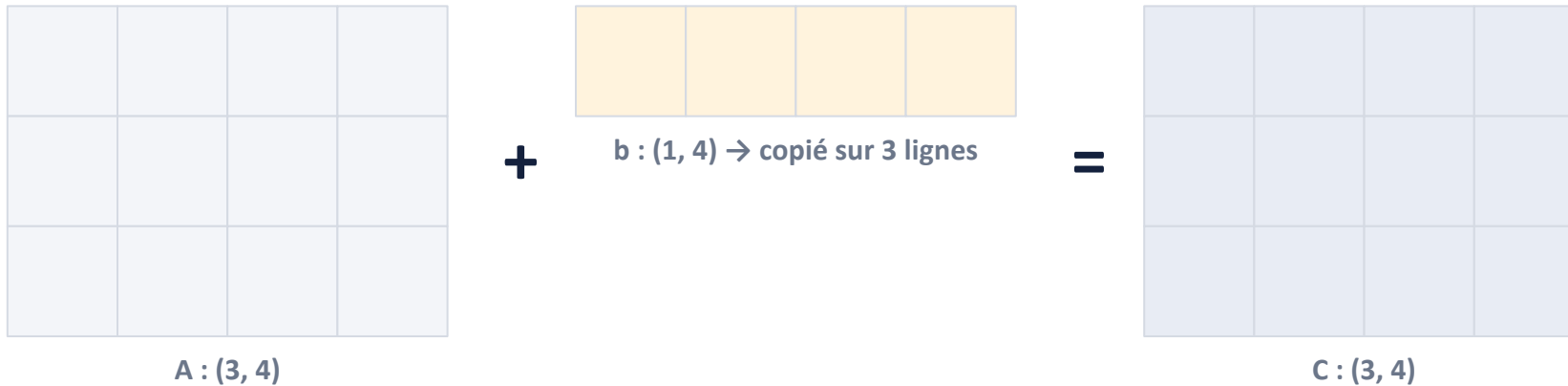
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def train(X, Y, alpha=0.05, epochs=1000):
    # X: (n_x, m), Y: (1, m)
    n_x, m = X.shape
    w, b = np.zeros((n_x, 1)), 0.0
    for _ in range(epochs):
        # seule boucle restante : le temps
        A = sigmoid(w.T @ X + b)
        # (1, m)
        J = -np.mean(Y * np.log(A) + (1 - Y) * np.log(1 - A))
        dZ = A - Y
        # (1, m)
        dw = X @ dZ.T / m
        # (n_x, 1)
        db = dZ.mean()
        w, b = w - alpha * dw, b - alpha * db
    return w, b, J
```

- @ = produit matriciel ; * = élément par élément.
- Tout est en **float64** par défaut : ajouter $1e-8$ dans les log en pratique.
- Le TP ajoute : historique du coût, prédiction, accuracy, comparaison des temps.

Broadcasting : NumPy étire les dimensions compatibles

Règle : on aligne les formes par la droite ; une dimension vaut 1 ou coïncide, sinon erreur



- $(3, 4) + (1, 4)$ ✓ $\cdot (3, 4) + (3, 1)$ ✓ $\cdot (3, 4) + (4,)$ ✓ $\cdot (3, 4) + (3,)$ ✗ ($3 \neq 4$ en alignant par la droite).
- Scalaire : $(m, n) + 1 \rightarrow 1$ est étiré partout. C'est ce qui rend `w.T @ X + b` valide avec `b` scalaire.
- Exemple utile : pourcentages par colonne `A / A.sum(axis=0, keepdims=True)` — `keepdims` garde la forme $(1, n)$.
- Le broadcasting est puissant **et** source de bugs silencieux : quand une forme surprend, `assert A.shape == (n, m)`.

Pièges NumPy : le vecteur « rang 1 »

python

```
a = np.random.randn(5)           # shape (5,) : ni ligne ni colonne
a.T.shape                         # (5,) : la transposée ne fait rien !
np.dot(a, a)                      # un scalaire (produit scalaire), pas une matrice

a = np.random.randn(5, 1)        # vecteur colonne explicite
a.T.shape                         # (1, 5)
np.dot(a, a.T).shape              # (5, 5) : produit externe, comme attendu

assert a.shape == (5, 1)          # documenter ET vérifier les formes
a = a.reshape(5, 1)              # reshape est O(1) : n'hésitez pas
```

- Toujours créer des vecteurs **(n, 1)** ou **(1, n)**, jamais **(n,)**, dans le code de ce cours.
- `reshape` est gratuit ; `assert` coûte rien et évite des heures de debug.
- Éviter `np.matrix` (obsolète) : uniquement `np.ndarray`.

NumPy avancé : la boîte à outils du data scientist

Besoin	Fonction	Exemple	Piège
Changer de forme	<code>reshape, .T, np.newaxis</code>	<code>X.reshape(m, -1).T</code>	-1 = « déduis la dimension »
Réduire sur un axe	<code>sum/mean/max(axis=...)</code>	<code>X.mean(axis=1, keepdims=True)</code>	axis=0 → le long des lignes
Empiler	<code>np.concatenate, np.stack</code>	<code>np.stack([a, b], axis=0)</code>	concatenate ne crée pas d'axe
Produits	<code>@, np.dot, np.einsum</code>	<code>np.einsum('ij,jk->ik', A, B)</code>	einsum : lisible mais lent en cas complexe
Sélection	masques booléens, <code>np.where</code>	<code>X[:, Y[0] == 1]</code>	renvoie une copie, pas une vue
Aléatoire	<code>np.random.default_rng(0)</code>	<code>rng.normal(0, 1, (n, m))</code>	fixer la graine pour reproduire
Numérique	<code>np.clip, np.log1p, np.exp</code>	<code>np.clip(A, 1e-8, 1-1e-8)</code>	log(0) = -inf casse le coût

6

Explorer les données et réduire la dimension

Avant d'apprendre une fonction, connaître les données : checklist d'exploration · PCA : intuition, algorithme, choix de k

Avant tout modèle : la checklist d'exploration

1 Forme et types

`shape`, `dtypes`, valeurs manquantes, doublons. Combien d'exemples par classe ? Déséquilibre ?

2 Distributions

Histogrammes par feature, échelles très différentes → standardiser. Valeurs aberrantes ?

3 Regarder les exemples

Afficher 20 images / 20 lignes au hasard. Étiquettes fausses ? Fuites (la réponse dans les features) ?

4 Corrélations

Matrice de corrélation, features redondantes → candidates à la PCA.

5 Baseline

Classe majoritaire, régression logistique : le score à battre, en 5 minutes.

6 Découpage

Train / dev / test **avant** toute décision, en respectant le temps ou les groupes si nécessaire.

PCA : projeter sur les directions de plus grande variance

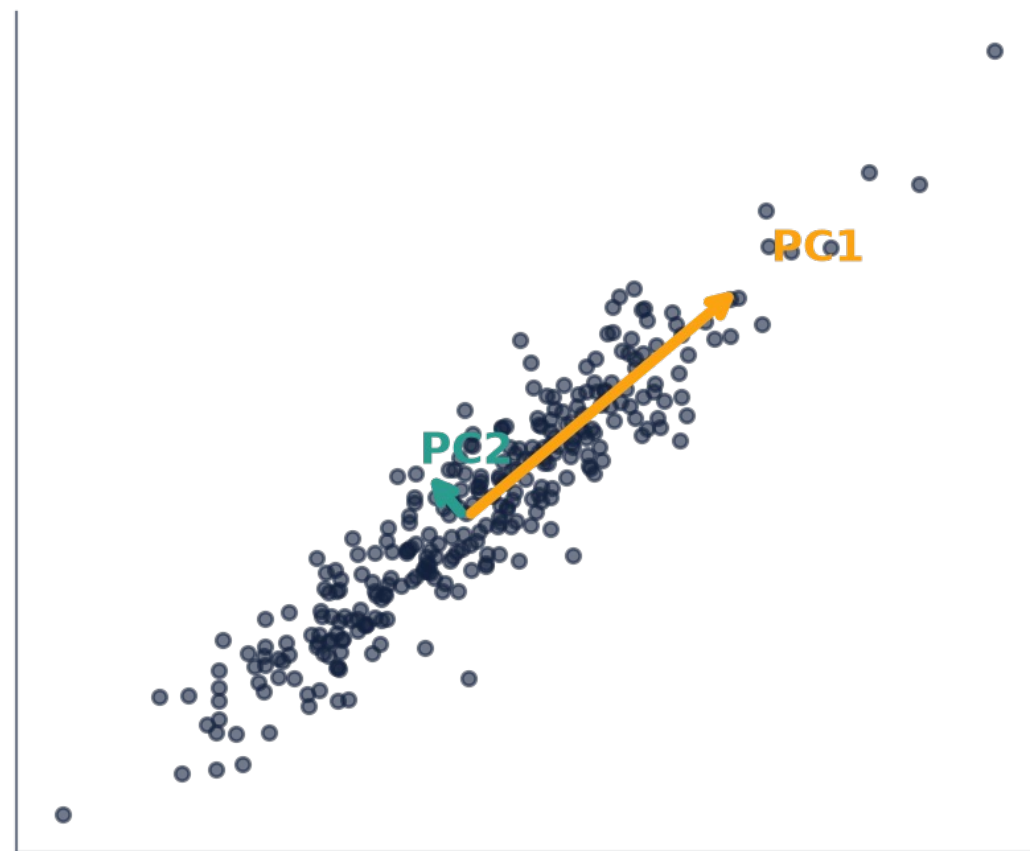
- Objectif : passer de n_x dimensions à $k \ll n_x$ en perdant le moins d'information (variance) possible.
- Les composantes principales sont **orthogonales** et ordonnées par variance décroissante.
- Usages : **visualiser** en 2D/3D, **décorrélérer** et compresser avant un modèle, **débruiter**.
- Limite : transformation **linéaire**. L'autoencodeur (J8) en est la version non linéaire apprise.
- Toujours **standardiser** avant : sinon la feature à grande échelle domine la première composante.

CONCRÈTEMENT

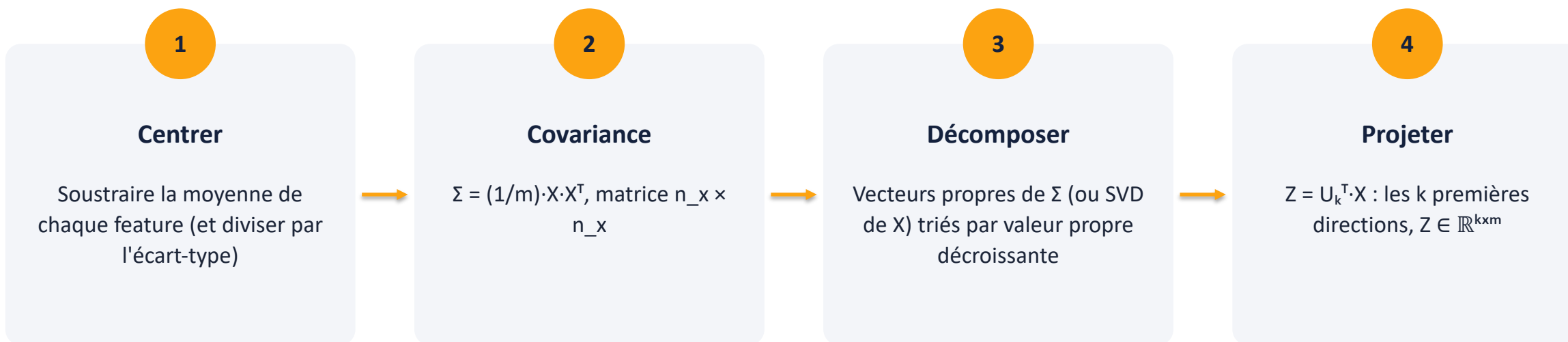
Rassembler des dizaines de colonnes en 2 axes, puis regarder si le dessin a du sens.

Exemple : on colorie les points chat / pas chat. Deux paquets séparés : bon signe. Tout mélangé : le problème sera dur.

PCA : axes de variance maximale



PCA : l'algorithme en quatre étapes

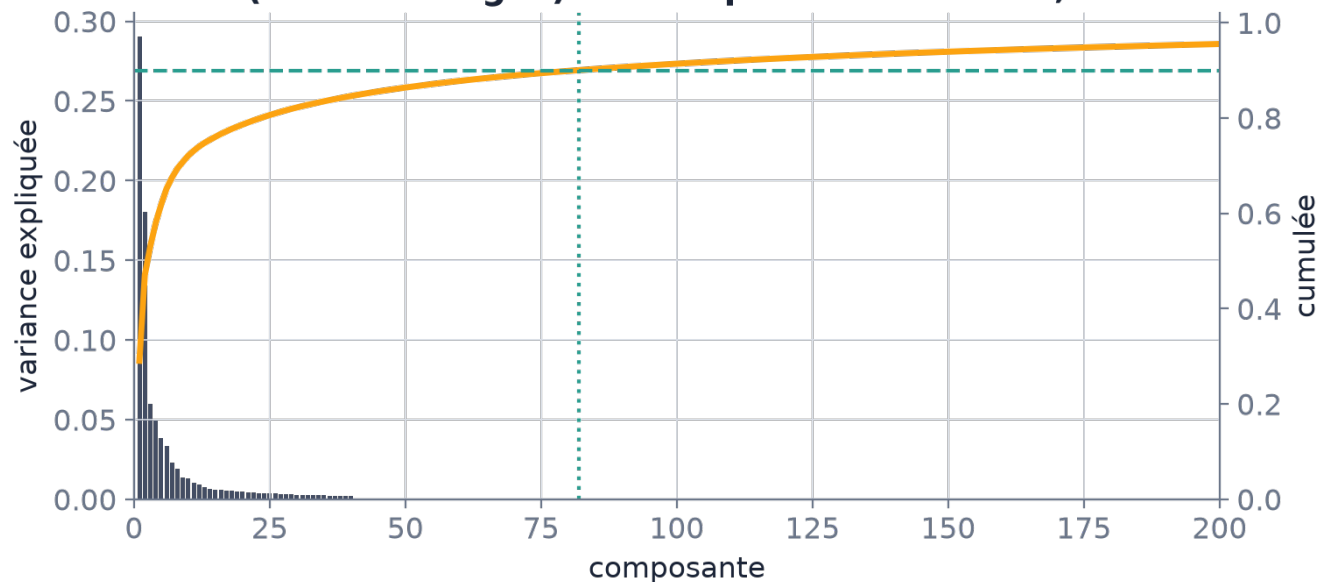


- Reconstruction : $\hat{X} = U_k \cdot Z$. L'erreur de reconstruction est la variance **non** expliquée.
- En pratique : `sklearn.decomposition.PCA(n_components=0.9)` garde 90 % de variance. Le TP demande aussi une version NumPy (`np.linalg.svd`).

Choisir k : le coude et la variance cumulée

- Tracer la variance expliquée par composante et son cumul.
- Règles usuelles : **90–95 %** de variance, ou le **coude** de la courbe.
- Pour visualiser : $k = 2$ ou 3 , quoi qu'en dise la variance.
- Fashion-MNIST : 784 pixels $\rightarrow$ 2 composantes séparent déjà grossièrement les classes ; il en faut ~ 80 pour 90 % de la variance.

Fashion-MNIST (10 000 images) : 2 composantes $\approx$ 47 %, 90 % vers $k \approx 82$



TP1 — NumPy avancé, exploration, régression logistique from scratch

OBJECTIFS

3h

- Manipuler des matrices sans boucle et mesurer le gain
- Explorer un jeu de données et le projeter par PCA
- Implémenter forward / coût / backward / descente de gradient

OUTILS

Python 3.11 · NumPy · Pandas · Matplotlib · scikit-learn (PCA) · Colab ou machine locale

NumPy avancé (45 min)

A

Reshape d'un batch d'images, broadcasting, `einsum`, benchmark boucle vs vectorisé, `assert` sur les formes.

Exploration + PCA (60 min)

B

Fashion-MNIST : distribution des classes, images moyennes par classe, PCA NumPy vs scikit-learn, projection 2D, variance cumulée.

Régression logistique (75 min)

C

Classifieur binaire « T-shirt vs pantalon » : sigmoid, coût, gradients vectorisés, courbe de coût, accuracy train/dev, effet de α .

Livrable — notebook `TP1_numpy_pca_logreg.ipynb` complété en séance, courbes incluses ; corrigé distribué en fin de TP.

Ce qu'il faut retenir de J1

- Un neurone = combinaison linéaire + activation ; un réseau = neurones empilés qui apprennent leurs features.
- Notation : $x^{(i)}$ en colonne, $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{1 \times m}$. Écrire les dimensions avant de coder.
- Régression logistique : $\hat{y} = \sigma(w^T x + b)$, perte logistique convexe issue du maximum de vraisemblance.
- Descente de gradient : $w \leftarrow w - \alpha \cdot dw$; $dz = a - y$; graphe de calcul et règle de la chaîne.
- Vectoriser : $Z = w^T X + b$, $dw = X \cdot dZ^T / m$. Broadcasting, jamais de vecteur $(n,)$.
- PCA : centrer, covariance, vecteurs propres, projeter ; choisir k par variance cumulée.

PROCHAINE ÉTAPE

J2 — 27/10 : réseaux à une puis L couches, backpropagation, puis le même modèle en PyTorch et Keras.



Pour aller plus loin : formules de référence

À relire chez vous ; rien ici n'est nécessaire pour suivre le TP

Le vocabulaire que nous utiliserons toute l'année

Mot	Définition en une phrase	Où on le rencontre
Modèle / architecture	La forme de la fonction (couches, connexions) ; l'architecture fixe la famille, les paramètres fixent l'instance	partout
Paramètres (W, b)	Les nombres appris par descente de gradient ; de 10^3 (TP) à 10^{12} (LLM)	J1–J2
Hyperparamètres	Les choix faits par l'humain : taux d'apprentissage, profondeur, taille de batch...	J5
Entraînement / inférence	Ajuster les paramètres sur des données / utiliser le modèle entraîné pour prédire	J1, J4
Perte (loss) / coût	L'erreur sur un exemple / sa moyenne sur le jeu ; ce que l'on minimise	J1
Métrique	Ce que l'on mesure pour décider (accuracy, F1, MAE) ; pas forcément la perte	J5
Epoch / batch / itération	Un passage complet sur les données / un sous-ensemble traité d'un coup / une mise à jour	J5
Représentation / embedding	Le vecteur intermédiaire calculé par le réseau, où les objets proches ont un sens proche	J2, J6, J9
Pré-entraînement / fine-tuning	Apprendre sur une grande tâche générale / adapter à la vôtre avec peu de données	J8, J9

Notation du cours (gardée jusqu'à la fin)

Un exemple

$$x^{(i)} \in \mathbb{R}^{n_x}, \quad y^{(i)} \in \{0, 1\}$$

exposant (i) : i-ème exemple

Le jeu de données

$$X = [x^{(1)} \ x^{(2)} \ \dots \ x^{(m)}] \in \mathbb{R}^{n_x \times m}$$

un exemple par colonne

Les étiquettes

$$Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}] \in \mathbb{R}^{1 \times m}$$

ligne de m valeurs

Couches (J2)

crochets : couche, chevrons : instant

$$a^{[l]}, W^{[l]}, b^{[l]} \quad ; \quad \text{séquences (J3)} : x^{\langle t \rangle}$$

Probabilités : de Bernoulli à la perte logistique

Modèle

y suit une loi de Bernoulli($\hat{y}$)

$$P(y = 1 \mid x) = \hat{y}, \quad P(y = 0 \mid x) = 1 - \hat{y}$$

Écriture compacte

$$P(y \mid x) = \hat{y}^y (1 - \hat{y})^{1-y} \quad \text{vaut } \hat{y} \text{ si } y = 1, 1-\hat{y} \text{ si } y = 0$$

Log-vraisemblance

maximiser $\Leftrightarrow$ *minimiser l'opposé*

$$\log P(y \mid x) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

Sur m exemples i.i.d.

$$\log \prod_i P(y^{(i)} \mid x^{(i)}) = \sum_i \log P(y^{(i)} \mid x^{(i)}) \quad \text{le log transforme le produit en somme}$$

Statistiques : ce qui sert tous les jours en deep learning

Moyenne, variance

$$\mu = \frac{1}{m} \sum_i x^{(i)}, \quad \sigma^2 = \frac{1}{m} \sum_i (x^{(i)} - \mu)^2$$

calculées **par feature** (axis=1 ici)

Standardisation

$$x \leftarrow \frac{x - \mu}{\sigma}$$

moyenne 0, variance 1 ; mêmes μ, σ sur le test !

Covariance

$$\Sigma = \frac{1}{m} (X - \mu)(X - \mu)^T \in \mathbb{R}^{n_x \times n_x}$$

base de la PCA (cet après-midi)

Espérance d'une moyenne

$$\mathbb{E}[\bar{x}] = \mu, \quad \text{Var}(\bar{x}) = \sigma^2 / m$$

un mini-batch = estimation bruitée ($J5$)